

Budgeting Time for Dynamic Vehicle Routing with Stochastic Customer Requests

Marlin W. Ulmer, Dirk C. Mattfeld, Felix Köster
Technische Universität Braunschweig, D-38106 Braunschweig, Germany,
m.ulmer@tu-braunschweig.de

August 26, 2015

Abstract

Parcel services route vehicles to pick up parcels in the service area. Pickup requests occur dynamically during the day and are unknown before their time of request. Due to working hour restrictions, vehicles are provided with a limited time budget to serve dynamic requests. As a result, not all requests can be confirmed. To achieve an overall high number of confirmed dynamic requests, dispatchers have to budget their time effectively anticipating future requests. The required time budget per customer depends on the instances, the current tour, and the time of day. An extension of the current tour allows a higher coverage of the service area and therefore a more efficient integration of future requests but reduces the free time budget. This results in a tradeoff between free time budget and area coverage. To determine how many requests can be confirmed in the future given a point of time, a tour and a free time budget, we present an anticipatory time budgeting heuristic (ATB). ATB draws on methods of approximate dynamic programming to evaluate the free time budget at a certain point of time regarding the expected number of future confirmations. Compared with state-of-the-art benchmark heuristics, ATB allows an effective use of the time budget resulting in anticipatory decision making and high solution quality.

keywords: dynamic vehicle routing, stochastic customer requests, time budget, subset selection, approximate dynamic programming, dynamic lookup table

1 Introduction

In last mile delivery, challenges for logistic service providers increase. Due to a significant increase in e-commerce sales, the number of shipped customer to customer (C2C) and business to customer (B2C) parcels has grown significantly. In Germany, the B2C and C2C market increased about 50% in the last five years (Esser and Kurte 2015). Many small vendors use online market places to sell their products directly to the purchaser. For successful e-commerce, delivery times and delivery costs are two of the main influence factors (Lowe et al. 2014). Customers expect reasonably priced, fast, and reliable service (Ehmke 2012). For fast delivery, about 20% of all parcels are shipped via courier or express delivery. Many of the parcels are picked up directly at the seller (in the following called *customer*) and are processed the same day (Hvattum et al. 2006). Some requests are known in the beginning but most of these pickups are requested in the course of the day (Lin et al. 2010). To serve these requests, courier express and parcel services schedule vehicles dynamically. The collected parcels are then transported to the depot to be shipped long haul (Pillac et al. 2013). In urban transportation, drivers' wages are the main cost factor. Since the drivers are already contracted for a daily working time, driving costs can be viewed as fixed (Thomas 2007). The beginning and the end of the working hours define the *shift* and result in a *time budget* for serving requests. Usually, the early

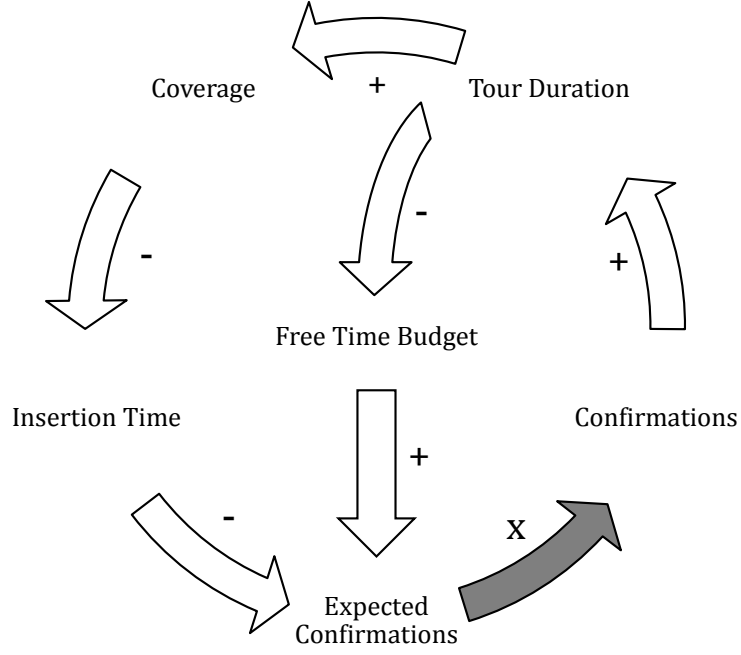


Figure 1: Influencing Factors on the Expected Number of Future Confirmations

request customers have to be served. Since the time budget is limited, not all dynamic requests can be confirmed. For every request, the dispatcher has to decide about a confirmation or rejection (Gendreau et al. 1999). Usually, the dispatcher accumulates requests until the vehicle reaches a customer’s location. Then, the subset of requests to be confirmed is selected (Meisel 2011). Rejected requests cause handling costs. They may be served by a third party, or may be postponed to following days (Angelelli et al. 2009). To include new requests, the dispatcher can dynamically adapt the planned tour if necessary (Gendreau et al. 2006). The duration of the planned tour has to be feasible regarding the time limit. Service providers aim on a high number of confirmations to maximize revenue and purchasers’ satisfaction and to reduce handling costs for the rejected requests.

Current decision making impacts the *expected number of future confirmations* (Bellman 1956). To allow maximizing the sum of immediate and future confirmations, future requests have to be considered in current decisions. For anticipation of future requests, the service provider can derive stochastic probabilities of customer requests for certain regions of the service area by making prognoses about customer behavior (Dennis 2011). Nevertheless, the requesting customers are arbitrarily distributed in the whole service area. Request times and locations are not known beforehand. Because of the high number of requests per day and the arbitrary customer locations, a straightforward consideration of possible future customers and their locations in decision making is not possible. In practice, dispatchers merely consider single future requests but aim on budgeting the limited time efficiently. Decisions depend on the required amount of time to insert a customer (*insertion time*), the remaining *free time budget*, and the current *point of time*. A high free time budget combined with a low insertion time leads to a high number of expected confirmations. Nevertheless, insertion time and free time budget are dependent. Figure 1 shows the influencing factors on insertion time and free time budget and their dependencies for a certain point of time. Notably, the point of time itself is another substantial influencing factor for the expected number of future confirmations. The factors and the fortitude of the depicted dependencies in Figure 1 are significantly influenced by the point of time.

The influence of a factor is indicated by an arrow. If the arrow is assigned to a plus symbol,

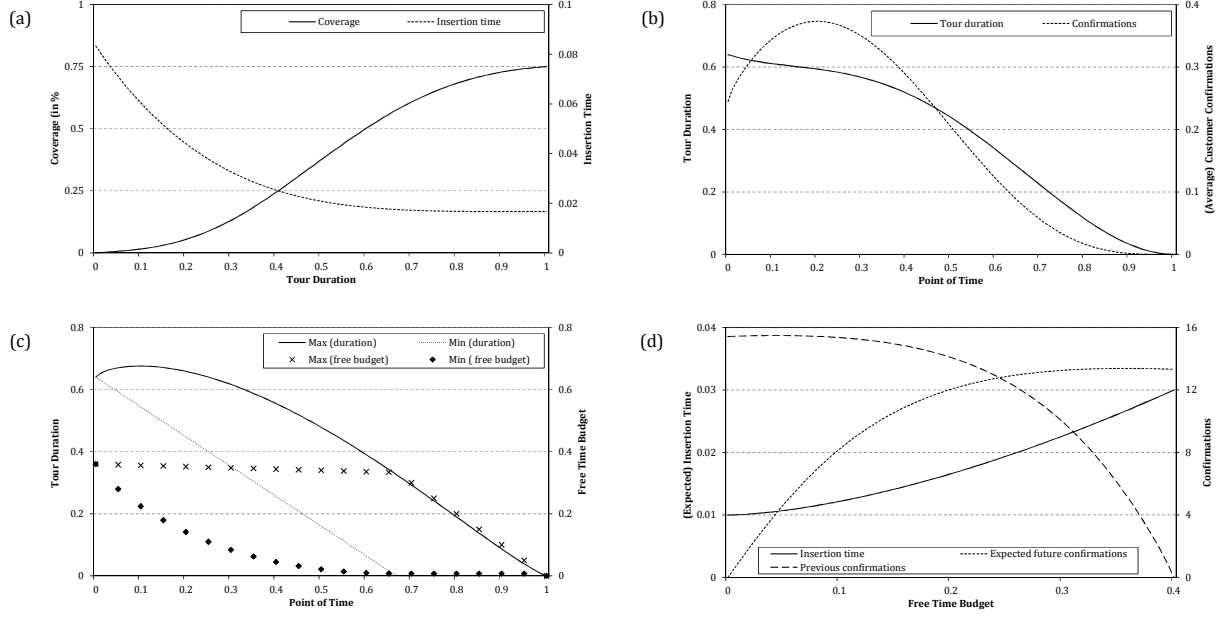


Figure 2: Impact of Coverage, Tour Duration, Insertion Time, Point of Time, and Free Time Budget on Expected Number of Future Confirmations

an increase in the influencing factor results in an increase in the influenced factor. The minus symbol indicates an opposed effect. The gray arrow assigned with an x reflects the position the dispatcher is able to control the dynamics. The dispatcher decides about the requests to confirm and the according routing considering the decision's impact to the expected number of future confirmations and the immediate confirmations. The applied decision results then in an adapted tour and a changed tour duration.

The insertion time significantly depends on the *coverage* of the service area (Branke et al. 2005). We define coverage as the percentage of the service area, where customer requests can be inserted in the tour with a reasonable insertion time, i. e., an insertion time lower than a certain threshold. An increase in coverage results in a decrease of the insertion time. A decrease in insertion time leads to an increase in expected number of future confirmations.

The coverage is dependent on the *tour duration* through the remaining early request and confirmed customers. If the tour duration increases, the coverage increases as well. As a result, a high number of immediate confirmations results in a long tour and low insertion time. This may increase the expected number of future confirmations.

Nevertheless, a high tour duration results in a low amount of free time budget. Even though the insertion time is low because of the high coverage, the expected number of future confirmations decreases. Dispatchers experience a tradeoff between tour duration, i. e., current confirmations and free time budget, respectively, future confirmations. To maximize the overall number of confirmations, dispatchers have to consider how a decision impacts the resulting insertion time and free time budget.

In Figure 2, we show how the dependencies impact the expected number of future confirmations in detail. For the purpose of presentation, we depict the dependencies of coverage, tour duration, insertion time, point of time, free time budget, and expected number of future confirmations in an exemplary and idealized way. Evidently, the dependencies are influenced by a variety of factors and are especially dependent on the instance settings.

The dependencies between the tour duration, the coverage, and the resulting insertion time are depicted in Figure 2(a). The tour duration $\bar{d}$ is depicted on the x-axis. $\bar{d} = 0$ represents

a tour with zero travel time, $\bar{d} = 1$ a tour requiring the whole time budget to conduct. $\bar{d} = 1$ is only feasible in the beginning of the shift resulting in no free time budget. The coverage is depicted on the left y-axis. 0 represents no coverage at all, while 1 represents full coverage of the whole service area. We assume that it takes a certain duration of the tour to establish an initial coverage. Then, the coverage increases simultaneously with the extension of the tour. Given a high tour duration, the increase subsequently diminishes. A full coverage is not achieved. The according insertion time relative to the overall time budget is shown on the right y-axis and is dependent on the coverage. The first extension of the tour reduces the insertion time significantly. This impact decreases if the tour duration is already high. As a result of these dependencies, we can utilize the tour duration as an indicator for the insertion time and coverage.

Figure 2(b) shows the average behavior of tour duration and confirmations over time assuming a given confirmation and routing policy of the dispatcher. The (average) tour duration dependent on the point of time t is depicted by the solid line. Point of time $t = 0$ represents the beginning of the shift, $t = 1$ represents the end. The customer confirmations over time are shown on the right y-axis. The dashed line shows the (average) confirmations at a certain point of time. In this case, the number of confirmations decreases over time. In $t = 0$, a tour is already given because of initial early request customers. The duration decreases with the number of customers already served and increases with new confirmations. Since dynamic requests are confirmed, the duration of the tour is kept almost constant for some time. Later, it decreases nearly linearly. Towards the end of the shift, the vehicle serves the remaining customers and returns to the depot. For this example, we assume a given decision policy about the confirmations and routing decisions. Nevertheless, decision making influences the tour duration significantly. The extrema are shown in Figure 2(c).

The point of time is depicted on the x-axis. On the left y-axis, the tour duration is shown. The right y-axis represents the according free time budget b , calculated by the point of time and the tour duration: $b = 1 - t - \bar{d}$. The dashed line shows the minimal $\bar{d}$ over time. This minimum is provided by a policy rejecting all dynamic requests. The vehicle constantly serves the early request customers and $\bar{d}$ decreases linearly. The vehicle arrives early at the depot. The solid line is the maximal possible $\bar{d}$ for every point of time. For the single points of time, this may be achieved by different policies.

Evidently, $\bar{d} \leq 1 - t$. The minimal and maximal amount of free time budget are depicted accordingly. As we can see, a corridor of free time budget is spanned for every point of time t .

For $t = 0.5$, Figure 2(d) shows this corridor on the x-axis. For this exemplary case, the maximal free time budget for $t = 0.5$ is $b = 0.34$ because the tour at least consists of the not yet served early request customers. The minimal free time budget is $b = 0$ meaning that all of the time budget is already consumed by previous confirmations. The expected insertion time for future requests, depicted on the left y-axis, directly depends on the free time budget resulting from the dependencies of free time budget and tour duration. Hence, the more free time budget is provided, the higher is the insertion time. Notably, the future insertion time differs over time remaining points of time and is dependent on the applied policy. The depiction in Figure 2(d) is a simplification. The free time budget is the result of previous confirmations before $t = 0.5$. Given $b = 0$, many requests are included. Some of them may be expensive, because they are not close to already existing customers and extend the tour duration significantly. The rejection of these requests leads to a substantial increase of the free time budget. For $b = 0.34$, all previous requests have been rejected. The expected number of future confirmations depends on the free time budget and the expected insertion time. Given $b = 0$, the insertion time is low, but no free time budget is remaining to insert a request. Given a high b , the insertion time increases significantly and the expected number of future confirmations stagnates. The dispatcher aims on maximizing the overall number of confirmations. If the expected number of future confirmations

per point of time and free time budget is provided in every decision point, an optimal decision can be selected maximizing the sum of immediate confirmations and expected future confirmations (Bellman 1956).

As a result of the presented dependencies, the point of time and the free time budget can be used as indicators for the expected number of future confirmations. Nevertheless, the presented dependencies are strongly influenced by the instance settings. E. g., a heterogeneous spatial request probability distribution in the service area impacts the dependency of tour duration and coverage. If requests cluster in certain regions of the service area, the number of future confirmations may depend on the vehicle’s and customers’ locations. Further, the coverage, tour duration, and according insertion times may change unpredictably, especially when a vehicle leaves a cluster (Meisel 2011, p. 210f). Hence, an analytical calculation of the expected number of future confirmations is not possible.

In this paper, we present an anticipatory time budgeting approach (ATB) approximating the expected number of future confirmations for a certain state of the problem. Therefore, we exploit the dependencies depicted earlier to differentiate states regarding the point of time and the free time budget. To determine how many future customer confirmations we can expect given a certain point of time and free time budget, we draw on approximate value iteration, an offline method of approximate dynamic programming (ADP, Powell 2011). Offline approaches allow efficient decision making within the problem’s execution phase. We represent states by vectors containing intervals of point of time and free time budget. This results in a two-dimensional lookup table. The length of the intervals is usually defined a-priori and significantly influences the approximation process in the learning phase (Fang et al. 2013). Since the factors behavior may differ over time as depicted earlier, a heterogeneous consideration of the time and the free time budget may be essential for accurate approximation. The according areas of time and free time budget may not be known beforehand, but depend on the instance settings. With the dynamic lookup table, we introduce an approach dynamically adapting the interval lengths regarding the instances and the approximation process to allow efficient and effective approximation.

We compare ATB with two benchmark heuristics. Present approaches for the presented problem generally aim for two directions. On the one hand, the routing is conducted explicitly considering the coverage of the service area resulting in a low insertion time (Branke et al. 2005). This is achieved by route selection and waiting-strategies. On the other hand, the subset of requests to confirm is selected explicitly. Feasible customers at inconvenient locations are rejected to maintain a high amount of free time budget (Meisel 2011). As depicted earlier, the outcomes of those two directions are highly interdependent. Each of the benchmark approaches mainly focuses on one of the two directions, coverage and subset selection. Anticipatory insertion (AI) aims on maintaining a high area coverage by idling at certain locations while confirming all feasible customer requests (Ghiani et al. 2012). The cost benefit heuristic (CBH) rejects inconvenient subsets by comparing the according time budget consumption with the number of confirmations (Ulmer et al. 2015). CBH does not explicitly consider the area coverage but decides about the selected subset. We compare ATB to the benchmark approaches for a variety of instances especially focusing on different distributions of customer locations. ATB is able to outperform both approaches increasing the number of confirmed requests significantly.

The contributions of the paper are aiming at both problem and solution methodology. This work transfers offline methods of approximate dynamic programming to vehicle routing with stochastic customer requests and unknown request locations. Until now, ADP was mainly successfully applied to problems with a small number of possible requests and known customer locations (Meisel 2011). Further, our presented approach ATB outperforms state-of-the-art benchmark heuristics and allows statements about the dependencies between the factors depicted in Figure 1. Finally, with the dynamic lookup table in § 4.5, we introduce a state space

Table 1: Literature Classification

	Problem Setting			Solution Approach		
	Objective	Representation	Nodes	Coverage	Subset	Anticipation
Psaraftis (1980)	time	distribution	-	e	-	i
Bertsimas and Van Ryzin (1991)	time	distribution	-	e	-	i
Tassiulas (1996)	time	distribution	-	e	-	i
Gendreau et al. (1999)	time	distribution	-	e	-	i
Swihart and Papastavrou (1999)	time	distribution	-	e	-	i
Ichoua et al. (2000)	time & cust.	distribution	-	e	i	i
Larsen et al. (2002)	time	distribution	-	e	-	i
Mitrović-Minić and Laporte (2004)	time	distribution	-	e	-	i
Thomas and White III (2004)	time & cust.	graph	126	e	i	e
Bent and Van Hentenryck (2004)	time & cust.	distribution	-	e	i	e
van Hemert and La Poutre (2004)	customers	graph	50	e	i	e
Branke et al. (2005)	customers	distribution	-	e	i	e
Ichoua et al. (2006)	time & cust.	distribution	-	e	i	e
Gendreau et al. (2006)	time	distribution	-	e	-	i
Hvattum et al. (2006)	time	distribution	-	e	-	e
Thomas and White III (2007)	time & cust.	graph	131	e	i	e
Thomas (2007)	customers	graph	50	e	i	e
Ghiani et al. (2009)	time	distribution	-	i	-	e
Meisel et al. (2009)	customers	graph	49	e	e	e
Meisel et al. (2011)	customers	graph	49	e	e	e
Ghiani et al. (2012)	customers	graph	50	e	i	e
Ulmer et al. (2015)	customers	distribution	-	i	e	i
<i>Anticipatory Time Budgeting</i>	customers	distribution	-	e	e	e

partitioning algorithm allowing effective and efficient approximate value iteration and simultaneously deriving insights of problem, instance, and solution structure.

This paper is outlined as follows. In § 2, we present and discuss the related literature focusing on vehicle routing problems with stochastic customer requests. In § 3, we formally define the dynamic routing problem. In § 4, we present the anticipatory time budgeting approach. Therefore, we briefly recall the functionality of ADP and introduce methods of state space aggregation and partitioning for an efficient approximation process. In § 5, we recall the benchmark heuristics AI and CBH. In § 6, conduct an extensive computational evaluation for a variety of real-world sized instances differing in customer distribution, service area size, and ratio of dynamic customers. The analysis of ATB and the according LTs in context of the dependencies depicted in Figure 1 is of particular importance for understanding how the approach works and how the factors are considered in the solution process. In § 7, we analyze the behavior of ATB regarding the the approximation process and these factors. The paper concludes with a summary of the results and directions for future research in § 8.

2 Literature Review

We consider a stochastic and dynamic vehicle routing problem based on the definitions of Kall and Wallace (1994). The problem is stochastic because not all information is provided in the beginning, but is revealed over time. It is dynamic because the problem setting allows adaptations of decision making regarding the revealed new information. The literature on stochastic and dynamic vehicle routing is vast. Stochastic impacts are uncertain travel times (Thomas and White III 2007, Lecluyse et al. 2009, Ehmke et al. 2015), service times (Daganzo 1984, Larsen et al. 2002), stochastic customer demands (Goodson et al. 2013) and requests (Psaraftis 1980, Bent and Van Hentenryck 2004). An extensive review of the different problem settings is given by Pillac et al. (2013). In the sequel, we focus on work considering stochastic customer requests. For these problems, decisions consider both routing and request confirmations.

Table 1 shows an overview of dynamic routing approaches with stochastic customer requests.

We classify work regarding the objective, the customer representation, and the solution approach. A part of the work aims on maximizing the number of served customers given a time limit. This work is represented by *customers* (or *cust.*) in the *Objective*-column. Some problems require to serve all customers minimizing the travel time, waiting time, number of vehicles, or the deviation from time windows. This is represented by *time*. For these problems, rejections are not possible.

In real-world routing problems, future requests generally follow a stochastic spatial-temporal *distribution*. Some problem definitions and instances base on a geographical aggregation (Campbell 2006) to simplify the problem structure. Possible future customers are represented by a set of nodes in a *graph* model. The vast number of possible customer locations given by the stochastic distribution is simplified to a set of known nodes. This bears the risk of a discrepancy between (aggregated) decision and practical implementation and, therefore, of inefficient solutions (Ulmer and Mattfeld 2013). The *Representation*-column of Table 1 indicates the customer representation. If potential customers are represented by a graph, the maximal number of nodes in the considered instances is shown in the *Nodes*-column.

We classify the applied solution approaches regarding their impacts on area *coverage* and *subset* selection. We differentiate between *implicit* (i) and *explicit* (e) impacts. If an approach aims on achieving sufficient area coverage to serve many future requests, the coverage is considered explicitly. If the approach actively decides about the *subset* to confirm allowing the rejection of feasible requests, it is called explicitly regarding the subset selection. Notably, approaches can consider both features explicitly, respectively implicitly.

We review the approaches regarding their degree of anticipating future requests. We distinguish between an (explicit) *anticipation* of future requests and (implicit) anticipation due to the decision selection without considering possible future requests. For an efficient classification, we call approaches considering future events explicitly anticipatory, all other approaches implicitly anticipatory. For an extensive overview of anticipation, the interested reader is referred to Schneeweiss (1999, p. 18ff).

The initial work on stochastic customer requests is aiming on applying plain routing heuristics to reduce the expected travel times. First come, first serve-policies are applied by Psaraftis (1980), Bertsimas and Van Ryzin (1991), Swihart and Papastavrou (1999), and Larsen et al. (2002). Tassoulas (1996) partitions the service region and subsequently serves the subareas. Gendreau et al. (1999, 2006) combine tabu search and an adaptive memory with a rolling horizon algorithm to dispatch customer requests to a fleet of vehicles. Ichoua et al. (2000) decide about routing given time windows and a time budget rejecting the resulting infeasible customers. Other approaches are straightforward waiting strategies (e.g., *wait at start*, *wait at end*), for instance applied by Mitrović-Minić and Laporte (2004). These approaches achieve implicit anticipation, but do not explicitly consider future requests in decision making.

Explicit anticipatory approaches can be divided in offline and online approaches. Offline approaches do not require extensive computational effort within the execution of the algorithm, but achieve a decision policy within a learning phase. For the given problem, offline algorithms are applied by Meisel et al. (2009) and Meisel et al. (2011). They use methods of approximate dynamic programming to evaluate post-decision states. Offline algorithms in vehicle routing generally suffer from the post-decision state space dimensionality because the values of post-decision states have to be stored for the execution phase (Powell 2011). As a result, Meisel et al. (2009) and Meisel et al. (2011) are only able to apply the approach for a graph containing 49 possible customer locations.

Explicit online anticipation is mainly achieved by sampling approaches. Within the execution phase, sampling approaches simulate a set of future events to evaluate current decisions. Sampling allows a more detailed consideration of future events, but requires significant computational effort within the execution phase. To anticipate stochastic customer requests in vehicle

routing, future customer requests are sampled according to the spatial distribution or the graph. These requests are used to evaluate different routes and decisions. Bent and Van Hentenryck (2003, 2004) introduce a sample-scenario planning approach, where future customer requests are sampled and integrated in plans containing a set of routes. This approach is also used by Flatberg et al. (2007) and Sungur et al. (2010). Ghiani et al. (2009) use short term sampling for a pick-up and delivery problem. Hvattum et al. (2006) approach a real-world case study with sampling. They use historical data of customer requests to minimize the expected travel time. Online sampling approaches consider subset selection only implicitly. Explicit subset selection would lead to an exponential increase of the already high calculation times and even to computational intractability (Powell and Ryzhov 2012, p. 203ff).

Thomas and White III (2004), van Hemert and La Poutre (2004), Branke et al. (2005), Ichoua et al. (2006), Thomas and White III (2007), and Thomas (2007) propose waiting heuristics including information about future requests to achieve explicit coverage consideration. These heuristics select the routing, waiting location, and waiting times according to potential future customers. Ghiani et al. (2012) introduce an anticipatory insertion waiting approach, where waiting considers the center of gravity of the potential and feasible future requests. Ghiani et al. (2012) compared AI with the sample-scenario planning approach by Bent and Van Hentenryck (2004) and achieved similar results, even though the sampling approach requires high computational effort. Therefore, we modify AI to benchmark our approach in § 5.

The majority of the approaches aims on providing a sufficient coverage and only implicitly decides about the subset to confirm. Generally, the largest feasible subset is included in the tour. Meisel et al. (2009), Meisel et al. (2011), and Ulmer et al. (2015) explicitly decide about the subset to confirm and include. Ulmer et al. (2015) propose a cost benefit heuristic comparing the relative insertion time of a subset candidate with the relative gain of confirming a subset. If the ratio exceeds a threshold, the candidate subset is rejected. The area coverage is considered only implicitly by Ulmer et al. (2015). To the best of our knowledge, CBH is the only approach allowing explicit subset selection for problem instances of real-world size. Therefore, we use CBH as a benchmark heuristic in § 5.

With ATB, we introduce the first offline approach evaluating post-decision states regarding their expected future confirmations, explicitly considering both the subset selection and the area coverage for problem settings of real-world size.

3 Problem Formulation

In the following, we present the formulation of the dynamic routing problem. To describe dynamic decision making, we use a Markov Decision Process (MDP, Bellman 1957). For a mathematical formulation of the ex-post mixed integer program, the reader is referred to Ulmer et al. (2015).

3.1 Problem Definition

A vehicle serves customer requests in a service area. It starts its tour at a depot and has to return to the depot within a given time horizon. In the beginning, a set of early request customers (ERC) is given. These customers have to be served. During the day, unknown late request customers (LRC) request service. Decision points occur by arriving at a customer. The dispatcher has to select the subset of new customer requests to confirm and the next customer to visit. Besides traveling, waiting at customer locations is permitted. Confirmations and rejections are permanent. Within each problem realization, the dispatcher aims on maximizing the number of served LRC.

Table 2: Problem Notations

Parameter	Notation
Time horizon	$T = [0, \dots, t_{\max}]$
Service area	$\mathcal{A}$
Depot	$\mathcal{D}$
Vehicle speed	v
Overall set of realizations	Ω
Realization	$\omega \in \Omega$
Customers of realization ω	$\mathcal{C}^\omega = \{C_1^\omega, \dots, C_h^\omega\}$
Early request customers	$\mathcal{C}_0^\omega$
Late request customers of a realization	$\mathcal{C}_+^\omega$
Spatial-temporal probability distribution	Ξ
Request time of customer $C_i^\omega \in \mathcal{C}^\omega$	$t_i \in T$
Location of customer $C^\omega \in \mathcal{C}^\omega$	$f(C^\omega) \in \mathcal{A}$
Travel time between customers $C_1^\omega, C_2^\omega \in \mathcal{C}^\omega$	$d(f(C_1^\omega), f(C_2^\omega))$
Decision points	$k = 0, \dots, K$
Initial state	S_0
State in decision point k	S_k
Decisions in decision point k	$X(S_k)$
Post-decision state in decision point k	S_k^x
Point of time in decision point k	$t(k)$
Vehicle position in decision point k	P_k
Customers to serve in decision point k	$\mathcal{C}_k$
Customer requests in decision point k	$\mathcal{C}_k^r = \omega_{k+1} \subset \omega$
Confirmed requests in decision point k	$\mathcal{C}_k^c$
Reward in decision point k	$\mathcal{R}_k$
Next customer to visit	C_{next}^k
Planned tour in decision point k , given decision x	Θ_k^x
Tour duration Θ_k^x	$\bar{d}(\Theta_k^x)$

The notation for the problem is depicted in the upper part of Table 2. In the lower part of Table 2, the terminology required for the Markov Decision Process is defined. Let $T = [0, 1, \dots, t_{max}]$ be the time horizon with time limit t_{max} . Let $\mathcal{A}$ be the service area. The vehicle starts and ends its tour in a depot $\mathcal{D} \in \mathcal{A}$. The vehicle travels with a monotone speed v . In the following, we describe a stochastic problem realization $\omega \in \Omega$ of the overall set of realizations Ω . Notably, in the decision making process, the customers are unknown before their request time. A problem realization $\omega \in \Omega$ consists of a set of customers $\mathcal{C}^\omega = \{C_1^\omega, \dots, C_h^\omega\}$, according requests times $t_i \in T$, and locations $f(C_i^\omega) \in \mathcal{A}, \forall C_i^\omega \in \mathcal{C}^\omega$ as depicted in Equation (1).

$$\omega = \{(C_1^\omega, t_1^\omega, f(C_1^\omega)), \dots, (C_h^\omega, t_h^\omega, f(C_h^\omega))\} \quad (1)$$

The customers are divided in two temporal classes as shown in Equation (2); early request customers $\mathcal{C}_0^\omega = \{C_i^\omega \in \mathcal{C}^\omega : t_i^\omega = 0\}$ requesting service in $t = 0$ and late request customers $\mathcal{C}_+^\omega = \{C_i^\omega \in \mathcal{C}^\omega : t_i^\omega > 0\}$ requesting service in $t > 0$. The customers $\mathcal{C}_0^\omega$ are known in the beginning and must be served.

$$\mathcal{C}^\omega = \mathcal{C}_0^\omega \cup \mathcal{C}_+^\omega = \underbrace{\{C_1^\omega, \dots, C_m^\omega\}}_{\mathcal{C}_0^\omega} \cup \underbrace{\{C_{m+1}^\omega, \dots, C_h^\omega\}}_{\mathcal{C}_+^\omega} \quad (2)$$

Each customer $C^\omega \in \mathcal{C}^\omega$ is assigned to a vector $(t^\omega, f(C^\omega))$ consisting of a request time $t^\omega \in T$ and a location $f(C^\omega) \in \mathcal{A}$. The assignment follows a spatial and temporal probability distribution $(t^\omega, f(C^\omega)) \sim \Xi : T \times \mathcal{A} \rightarrow [0, 1]$. The travel time between two customers is defined by $d(f(C_1), f(C_2))$.

Within each realization, the dispatcher aims on maximizing the number of confirmed late request customers. In unlikely cases, where the tour duration to serve all ERC already exceeds the time limit, all ERC are served and no confirmation of dynamic requests is allowed.

3.2 Decision Making

For the given problem, a decision contains both a confirmation and a movement action. To illustrate the decision making process, we model the problem as a Markov Decision Process. The required terminology is depicted in the lower part of Table 2. In an MDP, in each decision point k , a state S_k of a finite state space $\mathcal{S} = \{S_0, \dots, S_q\}$ and a subset of possible decisions $X(S_k) \subseteq X = \{x_1, \dots, x_r\}$ depending on state S_k is given. Each decision $x \in X(S_k)$ in state $S_k \in \mathcal{S}$ generates an immediate reward $R : \mathcal{S} \times X(S_k) \rightarrow \mathbb{R}$. The outcome of each combination $(S_k, x) \in \mathcal{S} \times X(S_k)$ is known beforehand and is defined as the post-decision state $S_k^x \in \mathcal{P} = \mathcal{S} \times X$. $\mathcal{P}$ denotes the post-decision state space. Given a post-decision state, a transition leads to a subsequent state $S_{k+1} = (S_k^x, \omega_k)$. S_{k+1} is determined by realization $\omega_k : \mathcal{P} \rightarrow \mathcal{S}$ of the set of (stochastic) problem realizations $\omega_k \in \Omega$.

For the given problem, the initial state S_0^ω is defined by the set of ERC $\mathcal{C}_0^\omega$ of the realization ω . A decision point occurs when the vehicle is located at a customer or initially at the depot. A state at decision point k consists of the point of time $t(k) \in T$, the vehicle's position $P_k \in \mathcal{A}$, and a set of customers to visit $\mathcal{C}_k^\omega = \mathcal{C}_0^\omega(k) \cup \mathcal{C}_+^\omega(k)$ containing the not yet served subset of ERC $\mathcal{C}_0^\omega(k) \subseteq \mathcal{C}_0^\omega$ and the not yet served subset of confirmed LRC $\mathcal{C}_+^\omega(k) \subseteq \mathcal{C}_+^\omega$. Additionally for $k > 0$, a set of requests $\omega_{k-1} = \mathcal{C}_r^\omega(k) \subseteq \mathcal{C}_+^\omega$ occurred between the last decision point $k - 1$ and decision point k is given.

Decisions $X(S_k)$ contain the confirmation action selecting the subset $\mathcal{C}_c^\omega(k) \subseteq \mathcal{C}_r^\omega(k)$ to confirm. As a movement action, the next customer $C_{next}^k \in \mathcal{C}_0^\omega(k) \cup \mathcal{C}_+^\omega(k) \cup \mathcal{C}_c^\omega(k) \cup \{P_k\}$ to visit is selected. If $C_{next}^k = P_k$, the vehicle idles at its current location for one time step $\bar{t}$. The idle time can be extended to w time steps by repeatedly setting $C_{next}^{k+j} = P_{k+j}$ for $0 \leq j < w$. The

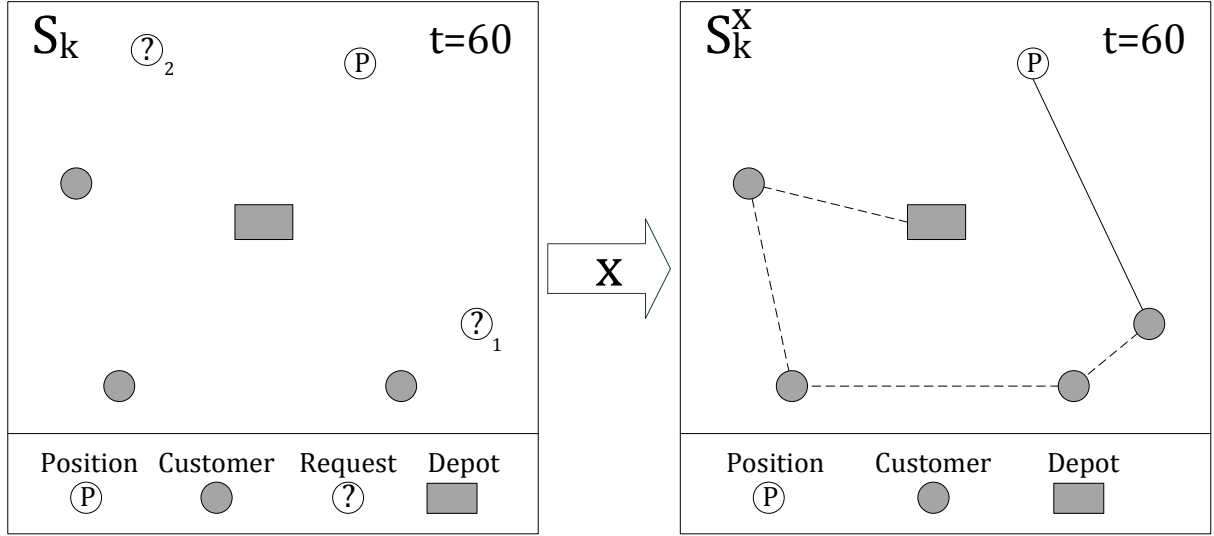


Figure 3: State S_k before and Post-Decision State S_k^x after Application of Decision x

immediate reward $R(S_k, x) = |\mathcal{C}_c^\omega(k)|$ of a decision x given state S_k is defined by the cardinality of the confirmed subset of requests.

A decision x is feasible if there exists at least one feasible tour $\Theta_k^x = (\theta_1, \dots, \theta_o)$ starting at the vehicle's position $\theta_1 = P_k$, traveling to the next customer $\theta_2 = C_{\text{next}}^k$, including all remaining confirmed late request and early request customers, and ending at the depot ($\theta_o = \mathcal{D}$). The tour duration Θ_k^x is defined as $\bar{d}(\Theta)$ in Equation (3).

$$\bar{d}(\Theta_k^x) = \sum_{i=1}^{o-1} d(f(\theta_i), f(\theta_{i+1})) \quad (3)$$

A planned tour Θ_k^x is feasible if $\bar{d}(\Theta_k^x) \leq t_{\max} - t(k)$, i. e., the tour duration of Θ_k^x allows returning to the depot within the time limit $t_{\max}$. A decision x results in a post-decision state S_k^x containing the time $t(k)$, the vehicle's location P_k , the next customer to visit C_{next}^k , and a set of remaining customers $\mathcal{C}_k^\omega \cup \mathcal{C}_c^\omega(k)$.

The travel to the next customer and realization ω_{k+1} lead to a transition to decision point $k+1$. State S_{k+1} consists of position $P_{k+1} = C_{\text{next}}^k$, time $t(k+1) = t(k) + d(f(P_k), f(C_{\text{next}}^k))$, and the set of customers $\mathcal{C}_{k+1}^\omega = \mathcal{C}_k^\omega \cup \mathcal{C}_c^\omega(k) \setminus \{C_{\text{next}}^k\}$. Further, ω_{k+1} provides a set of new requests $\omega_{k+1} = \mathcal{C}_r^\omega(k+1) = \{C_i^\omega \in \mathcal{C}_+^\omega : t(k) < t_i \leq t_{k+1}\}$. Because customer requests are stochastic, $\mathcal{C}_{k+1}^r$ is only revealed at decision point $k+1$ and unknown before. The overall realization $\omega \in \Omega$ is split regarding the time of the decision points as exemplarily depicted in Equation (4).

$$\omega = \underbrace{\{(C_1, t_1, f(C_1^\omega)), (C_2, t_2, f(C_2^\omega))\}}_{\omega_0}, \underbrace{(C_3, t_3, f(C_3^\omega))}_{\omega_1}, \dots, \underbrace{(C_{h-1}, t_{h-1}, f(C_{h-1}^\omega)), (C_h, t_h, f(C_h^\omega))}_{\omega_{K-1}} \quad (4)$$

The process terminates in state S_K , when the vehicle has returned to the depot, i. e., $\mathcal{C}_K^\omega = \emptyset$ and $P_K = \mathcal{D}$.

In Figure 3, an exemplary state S_k is shown. In $t = 60$, a set of three customers $\mathcal{C}_k^\omega = \{C_1, C_2, C_3\}$ has to be served. Two new requests $\mathcal{C}_r^\omega(k) = \{C_1^r, C_2^r\}$ are given. Decisions consist of four confirmation actions and up to six movement actions. In this exemplary case, the free time budget does not allow to confirm both requests. The applied decision x consists of

confirmation action $\mathcal{C}_c^\omega(k) = \{C_1^r\}$, i. e., to accept request C_1^r and to reject request C_2^r . This leads to an immediate reward of $R_k(S_k, x) = 1$. Further, the next customer to visit C_{next}^k is set to C_1^r . The resulting post-decision state is depicted on the right side of Figure 3. S_k^x contains four customers, the next customer to visit is depicted by the bold line. The dashed line indicates a feasible tour Θ_k^x .

The dispatcher is aiming for a high number of confirmations, i. e., a high overall reward. Because the customer requests are stochastic, future rewards are not known beforehand. The objective is to find an optimal decision policy $\pi^* \in \Pi$. A policy $\pi : \mathcal{S} \rightarrow \mathcal{X}$ is a sequence of decision rules, assigning every state $S_k \in \mathcal{S}$ to a decision $X_k^\pi(S_k) \in \mathcal{X}(S_k)$. $X_k^\pi(S_k)$ is the decision rule dependent on state S_k induced by π in decision point k . An optimal policy π^* maximizes the expected number of confirmations for an initial state S_0 , i. e., the expected rewards over all decision points applying π^* as depicted in Equation (5).

$$\pi^* = \arg \max_{\pi \in \Pi} \mathbb{E} \left[\sum_{k=0}^K R(X_k^\pi(S_k)) | S_0 \right] \quad (5)$$

Given a state S_k , π^* holds the Bellman Equation as depicted in Equation (6). In every decision point k , π^* selects the decision maximizing the sum of immediate reward and the expected future rewards.

$$X_k^{\pi^*}(S_k) = \arg \max_{x \in \mathcal{X}(S_k)} \left\{ R(S_k, x) + \mathbb{E} \left[\sum_{j=k+1}^K R(S_j, X_j^{\pi^*}(S_j)) | S_k \right] \right\} \quad (6)$$

4 Anticipatory Time Budgeting

For the presented problem, an optimal policy could be achieved by stochastic dynamic programming, recursively calculating the expected rewards as depicted in Equation (6). For an extensive overview of stochastic dynamic programming, the interested reader is referred to Kall and Wallace (1994). Nevertheless, for problem instances of real-world size, the number of states, decisions, post-decision states, and transitions is vast. The expected future rewards cannot be calculated exactly, but can only be approximated. This approximation can be achieved by approximate dynamic programming (Powell 2011). We utilize the concept of ADP to achieve our anticipatory time budgeting heuristic.

To allow efficient decision making within the execution phase, ATB is designed as an *offline* approach. This means that the calculation of the expected future confirmations (also called *values*) is conducted in a learning phase and the achieved results can be efficiently applied within the execution phase. During the execution phase, ATB draws on the values allowing for fast decision making. To store the values, an aggregation of the high-dimensional post-decision state space is necessary. As illustrated in § 1, the expected number of future customers significantly depends on the point of time and the free time budget. Therefore, a post-decision state is aggregated to a 2-dimensional vector containing point of time and free time budget. To evaluate the vectors, each dimension is partitioned in intervals. Such a partitioning defines a lookup table. Each entry of the LT is assigned to a value. Since a direct calculation of the values for an entry is not possible, we estimate the values by approximate value iteration, a method of ADP.

The partitioning and the resulting lookup table have a significant impact on the effectiveness and efficiency of the approximation. With the dynamic lookup table, we introduce an dynamic partition method adapting the partitioning with respect to the approximation process in § 4.5.

ATB aims on anticipatory subset selection considering the according impact on the area coverage. To determine the realized movement action and the positions in the tour, where new requests are inserted, we use cheapest insertion routing (Rosenkrantz et al. 1974, CI) for ATB and the benchmark approaches as described in § 4.1.

4.1 Routing and Insertion Decisions

ATB focuses on an efficient use of the time budget. Routing and insertion of the requests are secondary and follow plain heuristics. To allow a comparison of the approaches, we use cheapest insertion for routing and insertion for ATB and the anticipatory benchmark approaches. CI has several advantages. At first, it reflects the routing methods applied in practice. Usually, the driver plans a sequence of the ERC in the beginning of the day and subsequently adds new customers to the existing tour. Further, the application of CI is efficient and allows fast feasibility checks and decision making within the execution phase. Given a set of new requests $\mathcal{C}_k^r$, all approaches evaluate every potential candidate subset to confirm regarding feasibility and rewards. This results in a high number of candidate tours. Here, an efficient routing algorithm is mandatory to allow dynamic decision making.

The tour is initialized and updated regarding the following procedure: In $t = 0$, CI starts with a pre-decision tour $\Theta_0 = (\mathcal{D}, \mathcal{D})$ only consisting of the depot. In every decision point k , a pre-decision tour Θ_k and a number of candidate subsets $\mathcal{C}_i^\omega(k) \subseteq \mathcal{C}_r^\omega(k), i = 1, \dots, 2^{|\mathcal{C}_r^\omega(k)|}$ of new requests is given. CI subsequently selects the cheapest request of the subset regarding the insertion time and adds it at the cheapest insertion position in the tour. For $t = 0$, all requests are confirmed, i. e., $\mathcal{C}_c^\omega(0) = \mathcal{C}_0^\omega$. This results in decision x and post-decision tour $\Theta_k^x = (\theta_0, \dots, \theta_o)$. If the selected movement action of x is not waiting, the first customer in the tour $\mathcal{C}_{\text{next}} = \theta_1$ is the next customer to visit. After the travel to the next customer, the visited customer is removed from Θ_k^x . This results in the pre-decision tour $\Theta_{k+1} = (\theta_1, \dots, \theta_o)$. If waiting is applied, the pre-decision tour Θ_{k+1} is identical to Θ_k^x . If no customers are left to serve, i. e., $\Theta_k = (\mathcal{P}_k, \mathcal{D})$, but free time budget is left, the vehicle idles at the current location for all approaches. If no time is left, the vehicle returns to the depot.

A candidate subset and the according decision are considered feasible if the resulting candidate post-decision tour does not exceed the time limit. Because the decision space is reduced, CI may reject candidate subsets, which are feasible regarding a different routing approach, e. g., an optimal traveling salesman solution.

4.2 Approximate Dynamic Programming

The expected number of future rewards is represented by the value of a post-decision state and depends on the applied policy. The value for any given policy π can be recursively calculated applying decision rule X_j^π in all subsequent decision points $j = k + 1, \dots, K - 1$ for all possible transitions given a post-decision state S_k^x . The achieved value is called the (system) value $V_s^\pi(S_k^x)$ of the regarding post-decision state depending on π as depicted in Equation (7).

$$V_s^\pi(S_k^x) = \mathbb{E} \left[\sum_{j=k+1}^K R(S_j, X_j^\pi(S_j)) | S_k \right] \quad (7)$$

As shown in Equation (6), the decision rule of an optimal policy π^* maximizes the sum of the immediate reward and the value of the post-decision state. Equation (6) can be reformulated as shown in Equation (8).

$$X_k^{\pi^*}(S_k) := \arg \max_{x \in X(S_k)} \left\{ R(S_k, x) + V_s^{\pi^*}(S_k^x) \right\} \quad (8)$$

Conversely, the assignment of specific values to all post-decision states defines a decision policy if Equation (8) is applied. ADP aims on finding a policy π^a close to an optimal policy. Instead of directly approximating a policy π^a to the optimal policy π^* , methods of ADP aim on approximating the values V^{π^a} to the expected number of future confirmations $V_s^{\pi^*}$. This can be achieved by approximately solving the according MDP. Because of the dimensionalities of the problem, reductions of the transition space, the decision space, and the state space are necessary. In the following, we show how the reductions are conducted for ATB.

For the presented problem, the decision space is high-dimensional. Decisions contain the subset to confirm and the next customer to visit, respectively waiting. For state S_k with $|\mathcal{C}_k| = o$ and $|\mathcal{C}_k^r| = m$, the decision space size $|X(S_k)| \leq 2^m \cdot (m+o)$ is the product of the number of subset candidates and the number of possible next customers to visit including waiting. To reduce the decision space, we apply a decomposition $\mathfrak{D} : \{X_i \subset X\} \rightarrow \{X_j \subset X\}$. Movement actions are conducted by CI as described in § 4.1. Because no explicit spatial information is utilized in the algorithm, ATB is not able to represent waiting locations. Hence, the waiting option is neglected. The number of considered decisions $|\mathfrak{D}(X(S_k))| \leq 2^m$ is significantly reduced. For the reduction of transition space and state space, we use approximate value iteration, state space aggregation, and partitioning, as described in the following.

4.3 Approximate Value Iteration

To approximately solve the MDP, we apply approximate value iteration within the learning phase (Powell 2011, p. 127ff). Therefore, we start with initial values V^{π_0} and a resulting policy π_0 . Then, we subsequently simulate a subset of the problem realizations $\bar{\Omega} \subset \Omega$. Within each simulation run i , for the given realization $\omega^i \in \bar{\Omega}$, we apply policy π_{i-1} (*exploitation*). In some cases, decisions may be selected randomly to force an *exploration* of the state space. After each simulation run i , we update the values with the realized confirmations $V_{\omega^i}(S_k^x)$ according to Equation (9). Parameter α defines the step size of the approximation process.

$$V^{\pi_i}(S_k^x) = (1 - \alpha)V^{\pi_{i-1}}(S_k^x) + \alpha V_{\omega^i}(S_k^x) \quad (9)$$

The final policy $\pi_{|\bar{\Omega}|}$ is then applied in the execution phase.

4.4 Post-Decision State Space Representation

Every value calculated in the learning phase has to be stored to access it in the execution phase. For the given problem, the number of post-decision states $S_k^x \in \mathcal{P}$ is high, because customer requests can occur at any point in the service area. Further, in approximate value iteration, the more frequent a value is accessed and updated, the more accurate is its approximation. If values are only accessed sparsely, the approximation and the solution quality might be impaired (Barto 1998, p. 193). Hence, the state space representation has to be of reasonable size to allow both efficient storage and effective approximation. As illustrated in § 1, the expected number of future confirmations mainly depends on the point of time and the free time budget. The free time budget $b(k)$ is the difference of remaining time and tour duration as depicted in Equation (10).

$$b(k) := t_{\max} - t(k) - \bar{d}(\Theta_k^x) \quad (10)$$

We use an aggregation $\mathfrak{A} : \mathcal{P} \rightarrow \mathcal{Q} \subsetneq \mathbb{N}^2$ to represent high-dimensional post-decision states $\mathfrak{A}(S_k^x) = p^k$ by 2-dimensional vectors $p^k = (t(k), b(k)) \in \mathcal{Q}$ only containing the numerical parameters point of time and free time budget. The resulting representation $\mathcal{Q}$ is defined in Equation (11).

$$\mathcal{Q} = \{\mathfrak{A}(S_k^x) : S_k^x \in \mathcal{P}\} \quad (11)$$

In the decision making, the value of a post-decision state can now be represented by the value of the vector p^k : $V_s^\pi(S_k^x) \approx V^\pi(\mathfrak{A}(S_k^x)) = V_q^\pi(p^k)$, with $V_q^\pi(p^k)$ the value of vector p^k . The use of $\mathfrak{A}$ results in a significantly smaller representation $\mathcal{Q}$. $\mathcal{Q}$ can be imagined as a 2-dimensional lookup table. In each entry of this table, the value of the according vector p^k is stored. During the learning phase, the LT-entries are updated. The entry values of a LT induce a decision policy regarding Equation (8).

Partitioned Lookup Table. If we assume a minutely representation of T , the cardinality of representation $\mathcal{Q}$ is still high. The attributes point of time and free time budget represented on a minutely basis result in a representation size of $|\mathcal{Q}| \leq \frac{1}{2}(t_{\max})^2$, because the sum of point of time $t(k)$ and free time budget $b(k)$ has to be smaller than or equal to $t_{\max} \geq t(k) + b(k)$. As it is shown by Ulmer et al. (2015), an approximation based on $\mathcal{Q}$ eventually results in sufficient solution quality. Nevertheless, the high number of entries results in an extensive learning phase with a high number of required simulation runs. A minute-by-minute consideration of point of time and free time budget may not be necessary and may even impede the approximation process and the solution quality as depicted in Barto (1998, p. 193).

To allow a more efficient and effective approximation, we use a partitioned LT $\mathcal{E}$. $\mathcal{E}$ is defined by a partitioning $\mathcal{I} : \mathcal{Q} \rightarrow \mathcal{E}$ grouping vectors $p_1, \dots, p_q \in \mathcal{Q}$ to a set represented by a LT-entry $\eta = \mathcal{I}(p_1) = \dots = \mathcal{I}(p_q)$. Usually, partitionings generate LTs with sets of static interval lengths. An exemplary LT-entry $\eta \in \mathcal{E}$ consists of a set of vectors within an interval of lengths l : $\eta = \{(t, b), (t+1, b), \dots, (t+l-1, b), \dots, (t+l-1, b+l-1)\}$. The resulting LT $\mathcal{E}_l$ is significantly smaller and may allow a faster approximation. Nevertheless, an a-priori definition of the intervals may impede the solution quality because heterogeneous states are assigned to the same value. For an exemplary analysis of the partitioning's impact, the interested reader is referred to the Appendix. A suitable interval length may differ given different instances and even within an instance. This means that the required level of time-consideration varies and every a-priori interval selection provides suboptimal approximation.

Weighted Lookup Table. To combine the advantages of different interval lengths $l_1, \dots, l_L$ and to allow differing levels of time-consideration, George et al. (2008) and Powell (2009) suggest to combine multiple partitionings $\mathcal{I}_1, \dots, \mathcal{I}_L$ and the resulting LTs $\mathcal{E}_1, \dots, \mathcal{E}_L$. Let $V^{l_i}(\mathcal{I}_i(p))$ be the value of $\eta_{l_i} = \mathcal{I}_i(p)$ in LT $\mathcal{E}_{l_i}$ of a weighted LT (WLT) $\mathcal{E}_{\text{WLT}} = \bigcup_{i=1}^L \mathcal{E}_{l_i}$. The overall value $V_{\text{WLT}}(p)$ is calculated as the weighted sum of the single LT-values with weights $w_{l_1}, \dots, w_{l_L}$ for every LT as depicted in Equation (12).

$$V_{\text{WLT}}(p) = \sum_{i=1}^L w_{l_i}(\mathcal{I}_i(p)) V^{l_i}(\mathcal{I}_i(p)) \quad (12)$$

The weights are calculated considering the experienced variance within the entries $\sigma_{l_i}^2(\mathcal{I}_i(p))$, the number of observations $N_{l_i}(\mathcal{I}_i(p))$, and the bias $\mu_{l_i}(\mathcal{I}_i(p))$ of entry $\mathcal{I}_i(p)$. The bias $\mu_{l_i}(\mathcal{I}_i(p)) = |V^{l_i}(\mathcal{I}_i(p)) - V^{l_1}(\mathcal{I}_1(p))|$ is defined by the difference between value $V^{l_i}(\mathcal{I}_i(p))$ and the value of the lowest partitioning level $V^{l_1}(\mathcal{I}_1(p))$. A formula for the weights calculating the total variation of the error and considering all three impacts is provided by Powell (2009) as depicted in Equation (13).

$$w_{l_i}(\mathcal{I}_i(p)) \propto \left(\frac{\sigma_{l_i}^2(\mathcal{I}_i(p))}{N_{l_i}(\mathcal{I}_i(p))} + \mu_{l_i}(\mathcal{I}_i(p))^2 \right)^{-1} \quad (13)$$

The weighting favors LTs with large intervals in the beginning for a fast first approximation. Later, LTs with small intervals are weighted higher to achieve a more differentiated evaluation. The weight w_{l_i} for a LT $\mathcal{E}_i$ is increased by a relatively small variance and bias and a large number of observations. In the beginning, the frequently visited entries in the LTs with large intervals allow a fast first estimation of the value function. During the subsequent approximation process, the weights of the more detailed LTs with small intervals increase because of the relatively small variance and bias. Further, WLT allows avoiding ineffective LT-areas. For instance, areas in the LT providing relatively low numbers of expected future confirmations are early excluded from the approximation process. Hence, the approximation is focused on the effective areas. WLT may allow a faster approximation in the beginning and high quality solutions in the end without any tuning necessary. Nevertheless, WLT leads to increased memory consumption due to the high number of entries. Instead of a single LT, L LTs of different partitioning levels are required.

4.5 Dynamic Lookup Table

To exploit the advantages of the WLT and combine those with an efficient approximation, we introduce a third concept: the dynamic LT (DLT). DLT adapts to the problem specifics regarding value variance and number of entry observations and only requires a single LT. Compared to WLT, this significantly reduces the memory consumption. To the best of our knowledge, a method of ADP dynamically partitioning a lookup table with numerical parameters regarding the entry observations and value deviation has not been presented in the literature. Nevertheless, the idea of dynamic state space partitioning is not new. A similar concept of dynamically changing the partitioning has been introduced by Bertsekas and Castañon (1989). For an infinite-horizon problem with $|\mathcal{S}| = 100$ and a known transition probability matrix $\mathbb{P}$, Bertsekas and Castañon (1989) partition states regarding their residuals and achieve faster approximation processes. The presented approach is motivated by Bertsekas and Castañon (1989). Because of the large state space and the unknown transition probability function, a direct transfer of this approach is not possible.

DLT adapts the partitioning $\mathcal{I}$ according to the approximation process. DLT $\mathcal{E}^0$ starts in a partitioning $\mathcal{I}^0$ with large intervals and subsequently decreases the interval lengths in some areas during the approximation process. In the beginning, a fast, coarse-grained approximation of the few initial entries is provided. During the approximation, the interval length is decreased for some "interesting" areas, i. e., an entry is separated in a set of new entries resulting in a dynamic partitioning $\mathcal{I}^j$ and LT $\mathcal{E}^j$ in simulation run j . Hence, the DLT allows a fine-grained approximation within these areas. For areas of no interest, the partitioning stays in the original design. An evolution of the DLT is exemplified in Figure 4. In the left, initial partitioning, the intervals are homogenous. During the simulation runs, some areas are considered in more detail, as seen in the central partitioning. In the final partitioning, seen on the right, the lower left area is highly separated and considered in detail, while the large interval lengths of the initial partitioning in the upper right corner remain. Beside the advantage of a fast and effective approximation process, DLT may also allow to derive insights of the problem and solution structure. Areas with a high separation may indicate important areas or usual patterns for the problem.

Entry Selection. An entry $\eta = \mathcal{I}^j(p)$ of partitioning $\mathcal{I}^j$ and DLT $\mathcal{E}^j$ can only be considered in more detail if a sufficient number of entry observations is given. Otherwise, the new entries may not be observed frequently enough to derive a reliable approximation. Therefore, we consider the number of observations $N(\eta)$ to decide whether we are able to consider η in more detail. Further, entries which are frequently observed have an essential impact on the entire approximation process. The evaluation of those entries has to be very accurate and a detailed

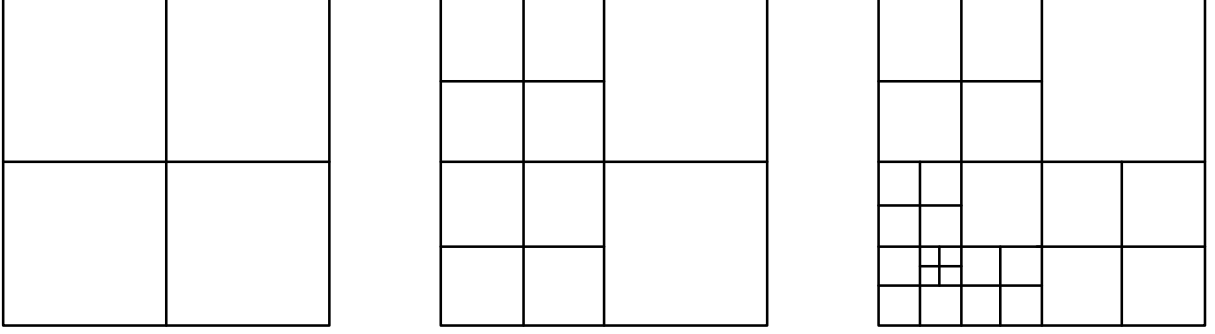


Figure 4: Exemplary Dynamic Evolution of a Partitioning $\mathcal{I}$ over the Approximation Process

consideration is desirable. A more detailed consideration of an entry is additionally required if states of heterogeneous value are grouped in a single entry. This is indicated by a high deviation within the entry's value, represented by the standard deviation $\sigma(\eta)$.

To achieve method independent of problem and instances, we use the relative values $\frac{N(\eta)}{\bar{N}}$ and $\frac{\sigma(\eta)}{\bar{\sigma}}$. $\bar{N}$ and $\bar{\sigma}$ are the averages of all entries $\eta_i \in \mathcal{E}^j$. A relatively high number of observations is indicated by $\frac{N(\eta)}{\bar{N}} > 1$, a relatively high standard deviation by $\frac{\sigma(\eta)}{\bar{\sigma}} > 1$. A separation of an entry η is conducted if Equation (14) is satisfied.

$$\frac{N(\eta) \sigma(\eta)}{\bar{N} \bar{\sigma}} \geq \tau \quad (14)$$

A multiplication of the two components allows deterministic entries without deviation to remain unseparated. Further, entries with only a few observations are not split allowing a reliable approximation. The tuning parameter τ defines the separation frequency of the DLT. A small τ results in a fast separation of many entries, a large τ only selects entries with outstanding characteristics in both number of observations and deviation. For comparison of the impact of N and σ to the approximation process, we define three DLT-approaches: $\text{DLT}(N, \sigma)$ considering both attributes and $\text{DLT}(N)$, $\text{DLT}(\sigma)$ only considering a single attribute in Equation (14).

Update Algorithm. In the update process of $\mathcal{I}^j$, entries $\eta = \mathcal{I}^j(p)$ satisfying Equation (14) are separated to a set of four new entries $\eta = {}_1\mathcal{I}_1^{j+1}(p), \dots, \eta_4 = \mathcal{I}_4^{j+1}(p)$ by dividing both intervals in half. Values, observations, and deviations are transferred to the new entries and the former entry is replaced. Because of the lack of further knowledge about the distribution of the values and observations, the number of observations and the deviation are equally divided to the new entries and the values remain.

5 Benchmark Heuristics

In this section, we recall benchmark heuristics of anticipatory insertion by Ghiani et al. (2012) and cost benefit by Ulmer et al. (2015). The approaches draw on the routing and insertion presented in § 4.1 differing regarding the management of the time budget and the subset selection. AI uses a part of the free time budget to wait at promising locations in $\mathcal{A}$ to maintain a high coverage of the service area and to efficiently include future requests. For AI, we implement different strategies to select the waiting points. CBH preserves the free time budget by rejecting requests with insertion times above average. Therefore, CBH compares the reward of including a candidate subset of requests with the required consumption of the free time budget. If the ratio of the insertion time exceeds the ratio of the rewards, the candidate subset is rejected. We select the approaches because they are state-of-the-art and focus on the two different decision aspects;

area coverage and subset selection. Further, they do not require high calculation effort within the execution phase. Dispatchers generally have only limited time to respond to customers' requests.

Beside AI and CBH, we apply a *myopic* heuristic to evaluate the approaches regarding anticipation. In every decision point, *myopic* selects the largest feasible subset of new requests. In cases, where several largest subsets are feasible, *myopic* selects the subset with the smallest consumption of the time budget. As movement actions, *myopic* neglects waiting and travels to the next customer in the tour. As a result, *myopic* may finish the tour early.

5.1 Anticipatory Insertion

AI was introduced by Ghiani et al. (2012) and draws on waiting strategies by Thomas (2007). For a dynamic routing problem with known customer locations, AI is able to achieve similar results as the sample-scenario approach by Bent and Van Hentenryck (2004). AI requires significantly less calculation effort in the execution phase and maintains the sequence of confirmed customers throughout the execution of the tour because of CI routing. The main idea of AI is to idle at certain locations in the service area to maintain a high coverage of the service area and, therefore, to insert new requests efficiently. AI draws on the *myopic* confirmation action.

To determine at which locations to wait, Ghiani et al. (2012) use the center-of-gravity (COG) longest wait strategy calculating the COG of all feasible potential future customers. The vehicle waits at the customer location, which allows the latest departure time serving the remaining customers and a dummy customer located at the COG. The COG is recalculated in every decision point. We call this approach AI_{COG}. For the presented problem, potential future customers are not known. To apply AI_{COG}, in every decision point, we sample a sufficient number of future customers. Then, we calculate the COG of the feasible sampled customers and proceed as described in Ghiani et al. (2012).

Beside AI_{COG}, we additionally combine AI with wait-at-start (AI_{WAS}) and wait-at-end (AI_{WAE}) strategies, also applied by Thomas (2007). In AI_{WAS}, the vehicle idles at the depot and confirms customer requests until the entire free time budget is consumed. In AI_{WAE}, the vehicle waits at the last customer in the tour until the free time budget is consumed or new requests are confirmed.

5.2 Cost Benefit

CBH originates from the idea of Ichoua et al. (2000) weighting travel time against resulting reward to decide about a diversion of the current tour. Ulmer et al. (2015) transfer this idea to decide about the acceptance of candidate subsets. Subset candidates requiring a relatively high consumption of the free time budget are rejected. As a result, CBH conserves a high free time budget to include future customers, but on the expense of less immediate confirmations and a suffering of the service area coverage. Given a tour Θ_k and a candidate subset $\mathcal{C}_k^c$, CBH compares the insertion time ("cost") of decision x inserting $\mathcal{C}_k^c$ with the rewards ("benefit") as depicted in Equation (15).

$$\kappa \cdot \frac{|\Theta_k^x|}{|\Theta_k|} \geq \frac{\bar{d}(\Theta_k^x)}{\bar{d}(\Theta_k) + d^*} \quad (15)$$

On the left side of Equation (15), the relative benefit, i. e., the increase in customers to serve caused by the confirmed requests of a decision is calculated. On the right, the duration of the new tour compared to the current tour is calculated. The parameters κ and d^* allow tuning regarding the instances. The average insertion time of adding a customer is dependent on the instances. Parameter κ scales the benefit compared to the costs. Parameter d^* defines a

free amount of time budget relative to the tour length. This enables the insertion of customer requests especially in the beginning, when the tour might be short and insertion times are above-average.

6 Computational Evaluation

In this section, we evaluate the approaches. We first define the set of test instances and tune the algorithm parameters. Then, we present the results of the ATB and benchmark heuristics for the defined instances.

6.1 Instances

The generation of the instances and the according parameters are described in the following.

The closed service area $\mathcal{A} \subseteq \mathbb{R}^2$ is rectangular defined by the points $(0, 0)$ and $(x_{\max}, y_{\max}) \in \mathbb{R}^2$. Time is represented minute by minute, $T = \{0, 1, \dots, t_{\max}\}$. The expected number of overall customers is $n = \mathbb{E}_{\omega \in \Omega} |\mathcal{C}^\omega|$. The expected number of early request customers $\mathbb{E}|\mathcal{C}^0| = n_0$ depends on the degree of dynamism $dod \in [0, 1]$ as depicted in Equation (16) (Larsen et al. 2002).

$$n_0 = (1 - dod) \cdot n \quad (16)$$

The number of customers and the request times for a realization are generated by a Poisson process $\mathfrak{P}$ (Haight 1967). The number of ERC is generated by $\mathfrak{P}(n_0)$. The probability distribution Ξ for request times and locations is divided into two independent probability distributions. Request times of late request customers are (discretely) uniformly distributed over time $t \sim U_{\mathbb{Z}}[1, t_{\max} - 1]$. Customer locations $f(C^\omega) \in \mathcal{A}$ are realizations $f \sim \mathcal{F}$ of the spatial probability distribution $\mathcal{F} : \mathcal{A} \rightarrow [0, 1]$.

A realization of the request time is again conducted by a Poisson process $\mathfrak{P}$ for every minute $0 < t < t_{\max}$. Given two points of time $0 < t^j < t^h < t_{\max}$, this results in an expected number of customers of $n_{t^j}^{t^h} = \mathbb{E}_{\omega \in \Omega} |\{C_i^\omega \in \mathcal{C}_+^\omega : t^j < t_i \leq t^h\}|$ requesting in time $t^j < t_i \leq t^h$ as described in Equation (17).

$$n_{t^j}^{t^h} = dod \cdot n \cdot \frac{t^h - t^j}{|T| - 2} \quad (17)$$

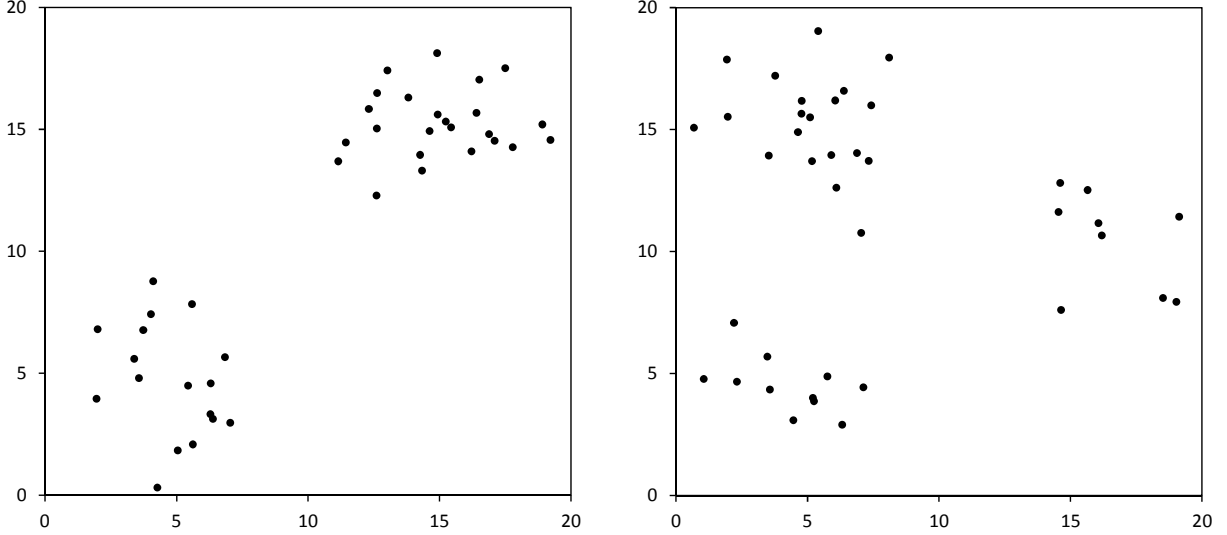
The travel time $d : \mathcal{A} \times \mathcal{A} \rightarrow \mathbb{N}$ for two customers $C_1^\omega, C_2^\omega \in \mathcal{C}^\omega$ with locations $f(C_1^\omega) = (a_1^x, a_1^y), f(C_2^\omega) = (a_2^x, a_2^y) \in \mathcal{A}$ is Euclidean and rounded up to minutes as depicted in Equation (18). The minimal travel time is set to $\bar{t} = 1$ minute.

$$d(f(C_1^\omega), f(C_2^\omega)) = \max \left(\left\lceil \frac{((a_1^x - a_2^x)^2 + (a_1^y - a_2^y)^2)^{1/2}}{v} \right\rceil, 1 \right) \quad (18)$$

The quantities of the instances are derived from Bent and Van Hentenryck (2004), Hvattum et al. (2006), Thomas (2007), and Meisel (2011). The number of customers is increased to match real-world quantities. The instance parameters are listed in Table 3. The time limit is set to $t_{\max} = 360$ minutes. We test the approaches for a large ($\mathcal{A}_{20} : x_{\max} = 20km, y_{\max} = 20km$) and small service area ($\mathcal{A}_{15} : x_{\max} = 15km, y_{\max} = 15km$). The vehicle travels with a speed of $v = 25km/h$. The depot is located in the center of the area $\mathcal{D}_{20} = (10, 10)$, respectively $\mathcal{D}_{15} = (7.5, 7.5)$. The average number of customer requests per day is $n = 100$. We examine instances with a low ($dod = 0.25$), moderate ($dod = 0.50$), and high ($dod = 0.75$) number of dynamic requests. We define three spatial distributions. We consider uniformly distributed

Table 3: Instance Parameters

Time limit	$t_{\max} = 360$ min
Expected number of customer	$n = 100$
Vehicle speed	$v = 25$ km/h
Service area	$\mathcal{A}_{15}, \mathcal{A}_{20}$
Degree of Dynamism	$dod = 0.25, 0.50, 0.75$
Spatial distribution	$\mathcal{F}_U, \mathcal{F}_{2C}, \mathcal{F}_{3C}$

Figure 5: Exemplary Realization of $\mathcal{F}_{2C}$ and $\mathcal{F}_{3C}$ for $\mathcal{A}_{20}$

customers ($\mathcal{F}_U$) and customer distributions grouped in two ($\mathcal{F}_{2C}$) or three clusters ($\mathcal{F}_{3C}$). Within the clusters, the customers are normally distributed. Given $\mathcal{A}_{20}$, an exemplary customer setting with 50 realized customers for $\mathcal{F}_{2C}$ and $\mathcal{F}_{3C}$ is shown in Figure 5. $\mathcal{F}_{2C}$ and $\mathcal{F}_{3C}$ are defined to analyze the impact of a heterogeneous spatial distribution to decision making, keeping in mind that ATB only draws on temporal attributes and neglects explicit spatial information. We assume that ATB is able to perform well for the symmetrical $\mathcal{F}_{2C}$, because the spatial information about the sequence of cluster visits does not influence the temporal attributes and the regarding outcomes. For $\mathcal{F}_{3C}$, the sequence significantly impacts the outcome of the time budgeting. For these instances, ATB decision making might be impaired.

In the following, we define the spatial distribution functions for $\mathcal{A}_{20}$. Given $\mathcal{F}_U$, a realization $f(C) = (a_x, a_y)$ is defined as $a_x, a_y \sim U[0, 20]$. For $\mathcal{F}_{2C}$, the customers are equally distributed to each cluster. The cluster centers are located at $\mu_1 = (5, 5), \mu_2 = (15, 15)$. The standard deviation within the clusters is $\sigma = 1$. The distribution is therefore point-symmetrical to the depot. For $\mathcal{F}_{3C}$, the cluster centers are located at $\mu_1 = (5, 5), \mu_2 = (5, 15), \mu_3 = (15, 10)$. 50% of the requests are assigned to cluster two, 25% to each other cluster. The standard deviations are set to $\sigma = 1$. For $\mathcal{A}_{15}$, all spatial parameters are reduced by factor 0.75. Unlikely distribution outliers outside of $\mathcal{A}$ are tolerated and handled regarding Equation (18).

6.2 Parameter Tuning

In the following, we describe the required tuning for ATB and the benchmark approaches.

ATB. We consider $L = 5$ different levels of partitioning, starting with intervals of length 16 ($l = 5$) up to the discrete value representation ($l = 1$). So, the time and time budget

consideration varies from 16 minutes down to minute by minute. We apply lookup tables with static interval length $SLT(I)$ for all levels $I = 2^{l-1}, l = 1, \dots, 5$. For the weighted and dynamic LTs, we consider all five partitioning levels ($l = 5$ up to $l = 1$). We apply DLTs with $\tau = 1.0, 1.25, \dots, 2.0$. For $\tau = 1.0$, an entry is separated if its divergence in observations and standard deviation is above average. For $\tau = 2.0$, the divergence has to be two times as high. For $\hat{V}_0$, we choose high initial values to force exploration in the beginning resulting in the selection of not yet observed entries. For all LTs, we run $|\bar{\Omega}| = 1$ million simulation runs. The derived policies are then used for the evaluation. To define the further specifics of the ADP algorithm, the interested reader is referred to the taxonomy provided by Powell (2007, p. 296ff). Powell differentiates seven characteristics to describe an ADP approach. For most of the characteristics, we stay with the "standard" design:

1. (c) State variable: The values are based on aggregated post-decision states.
2. (a, i) State sampling: States are sampled asynchronously. Pure exploitation is used.
3. (a) Average vs. marginal values: The average state values are estimated.
4. (a) State space representation: The values are stored in a LT. We use static, weighted, and dynamic partitionings.
5. (b) Value selection for update: We update the values at the end of each simulation run.
6. (a) Number of samples: We update the values after each realization.
7. (a, i) Update of the values: We smooth the new estimate with the current approximation calculating the moving average.

AI. To determine the COG, we sample a set of 100 spatial realizations $F \subset \mathcal{A}$ of the respective distribution $\mathcal{F}$. Each sampled customer is checked for feasibility in the current tour. This results in a subset of feasible realizations $\bar{F} \subset F$. If $|\bar{F}| = 0$, the COG is set to the depot. Else, the COG is calculated regarding Equation (19).

$$\text{COG} = \left(\sum_{(a_x, a_y) \in \bar{F}} \frac{a_x}{|\bar{F}|}, \sum_{(a_x, a_y) \in \bar{F}} \frac{a_y}{|\bar{F}|} \right) \quad (19)$$

CBH. To determine parameters d^* , κ , for every instance setting, we run 100 test runs within a learning phase. For every test run, we apply candidate parameter vectors (d_c^*, κ_c) with $d_c^* \in T$ and $\kappa_c \in [0, 2]$. We select the parameter combination maximizing the overall sum of confirmations for the execution phase.

6.3 Solution Quality

For every instance setting, we run 10,000 test runs and present the average percentage of served dynamic requests. The solution qualities for ATB drawing on the different LTs and the benchmark heuristics are shown in the upper part of Table 4. For the different instances, we focus on the parametrized LT providing the best solution quality (+) and the average solution quality ($\emptyset$) of the approach group. For each instance setting, the best result is printed in bold.

At first, we analyze the impact of the instance parameters on the solution quality. The number of served requests is dependent on the *dod*, the customer distribution, and the service area size. For varying *dod*, the number of served requests changes but the percentage remains constant. Given a small service area, more requests can be served due to shorter travel times.

For the clustered distributions, customers are located close to each other and no far-off ERC reduce the initial time budget. In contrast to $\mathcal{F}_U$, insertion times are relatively low. In the extreme case of $\mathcal{A}_{20}$, $\mathcal{F}_U$, and expected 75 ERC ($dod=0.25$), the initial tour already consumes the entire time budget. We exclude this instance in the following detailed analysis of the solution quality.

Comparing the solution approaches, the best results are achieved by ATB in 14 of the 17 remaining instance settings. ATB performs better given a high dod because the free time budget increases and the solution quality is less influenced by single customers and spatial information. Only given a low $dod = 0.25$, ATB is outperformed by AI. CBH achieves a sufficient number of confirmations and anticipation regardless the instances but is generally not able to achieve the same solution quality as ATB. The explicit subset selection of CBH avoids confirmations of inconvenient customers with high insertion times, but is not able to incorporate instance characteristics into decision making. The results of AI depend on the determination of the points of time the vehicle idles. While AI_{WAS} performs worse than the myopic approach, AI_{COG} and AI_{WAE} allow anticipation and improve the results compared to the myopic approach up to 8.6%. The results for AI_{WAE} always outperform AI_{COG} . Hence, waiting at later points of time is more effective. AI performs well if only a small number of dynamic customers is given. If the number of dynamic requests increases, waiting unnecessarily consumes time which cannot be used to serve (future) customer requests.

As assumed, in the three cases of $\mathcal{A}_{15}$, $\mathcal{F}_{3C}$, AI is able to outperform ATB because of the varying sequence of visited clusters. This spatial information is not considered by ATB. Hence, the solution quality is impeded. Given the symmetrical $\mathcal{F}_{2C}$, the cluster sequence is not relevant for later outcomes allowing ATB to achieve a high solution quality without considering spatial information.

ATB outperforms the *myopic* approach for nearly every instance. The percentage of improvement compared to the myopic approach is depicted in the lower part of Table 4. The best solutions gain an improvement up to 38.5%. The average improvement over all 17 instance settings is 9.3%. Especially for $\mathcal{F}_U$, the average improvement of 20.9% is high because the explicit subset selection of ATB avoids customers far-off the current tour. Generally, in cases where the number of confirmations by the myopic approach is low, anticipation allows far better solutions, e. g., for $\mathcal{A}_{20}$ or $\mathcal{F}_U$. Here, single, expensive confirmations significantly influence the rest of the planning period. In cases, where the number of accepted confirmations provided by the myopic solution is already high, the improvements are less distinct.

7 Analysis

In this section, we analyze the ATB algorithm. We compare the effectiveness and efficiency of the approximation process for ATB with DLT, WLT, and LTs with static interval lengths. For an exemplary instance, we analyze the structure of the DLT deducting statements about the achieved time budgeting decisions. We analyze the dependencies of tour duration, insertion times, expected values, and confirmation decisions over time.

7.1 Approximation Process

Figure 6 shows the ATB-approximation process for SLT, WLT, and $DLT(N, \sigma)$ given $\mathcal{F}_{2C}$, $dod = 0.75$, and $\mathcal{A}_{20}$. The x-axis represents the first 200,000 simulation runs. On the y-axis, the according solution quality is shown averaged over 10,000 simulation runs for a better presentation. We compare the tuned LTs providing the best solution quality after 1 million simulation runs and the average results of the tuning group. For this instance setting, DLT with $\tau = 1.5$ and

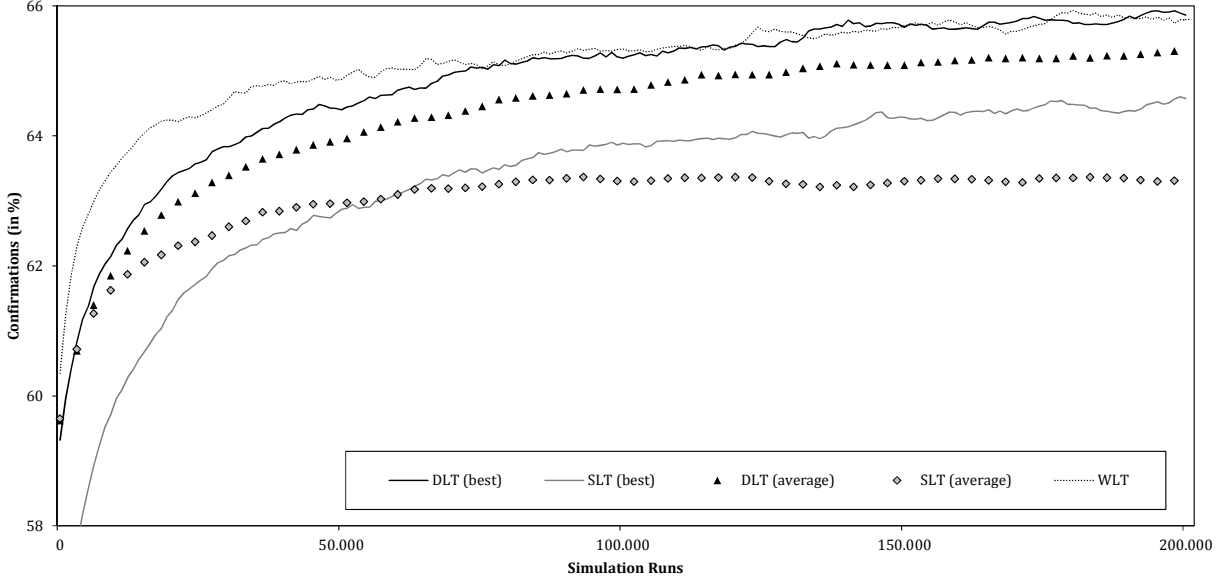


Figure 6: Approximation Process for $\mathcal{F}_{2C}$, $dod = 0.75$, and $\mathcal{A}_{20}$

SLT with $l = 2$ achieve the best solution quality. Comparing DLT and SLT, the average DLT allows a faster approximation than the best SLT. For this instance, the best approximation in the first 10,000 simulation runs is achieved by WLT converging to similar results as the best DLT given 75,000 simulation runs.

By comparing the best and average solution quality of the SLTs over all instance settings, we can detect a gap of up to 15.8% in confirmations. This confirms that the static and a-priori definition of the interval size has a significant impact on the solution quality as predicted. Even though SLTs may allow approximation for one interval size, for others the solution quality cannot exceed the *myopic* approach. The interval size providing the best solutions differs and depends on the number of simulation runs and instance settings. As expected, large interval lengths allow a fast approximation. Small interval lengths provide high solution quality on the expense of a high number of required simulation runs. The best solutions are provided by $I = 1, 2, 4$. For $I > 4$, the detail of time-consideration is too low to provide sufficient approximation.

The best results of the DLTs are achieved by $1.5 \leq \tau \leq 2.0$. For $\tau < 1.5$, the fast separation reduces the approximation speed slightly. The solution quality remains almost constant for the different τ . The average results of $DLT(N, \sigma)$ and $DLT(N)$ are always higher compared to the SLTs. The average solution quality of the DLTs provides up to 15.9% more confirmations than the average SLTs and, in some cases, even outperforms the best SLT. Except for one case, $DLT(N, \sigma)$ performs as well as or better than WLT. For 10 of the 17 instances, the best $DLT(N, \sigma)$ outperforms WLT and the average $DLT(N, \sigma)$ provides at least equal results.

Table 4 shows that the DLT considering only one aspect differs in solution quality and approximation process. While $DLT(N)$ achieves similar and for some instances even better results than $DLT(N, \sigma)$, $DLT(\sigma)$ is not able to provide high solution qualities for all instance settings. As expected, the separation of sparsely visited entries leads to an impaired approximation. Further, entries representing the beginning of the time horizon have a higher variance, because the stored values are higher compared to entries representing later times of the time horizon.

In the following, we analyze the structure of the achieved DLTs. We exemplarily select the instance setting with $\mathcal{F}_{2C}$, $dod = 0.75$, and $\mathcal{A}_{20}$, because the instance structure allows a vivid display of the results and distinguished conclusions. Figure 7 shows the value development regarding the point of time for a fixed budget of $b = 100$ for WLT on the left and the DLTs on

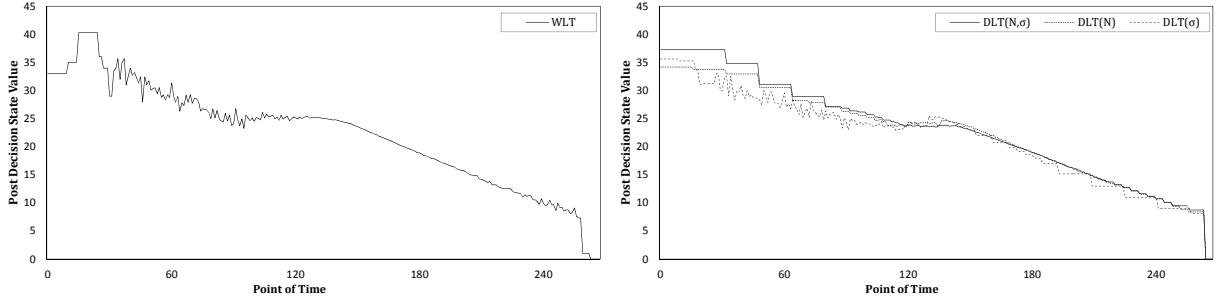


Figure 7: Value over Time given Time Budget $b = 100$ for $\mathcal{F}_{2C}$, $dod = 0.75$, and $\mathcal{A}_{20}$

the right side.

At first, we analyze the development of the WLT-values on the left side. In the beginning with $t \leq 90$, the time budget generally exceeds 100 minutes, so only a few observations are made. Between $90 \leq t \leq 120$, a plateau in the value development can be observed and only a few decision points occur. The values remain constant between $90 \leq t \leq 120$ indicating a low number of confirmations within this time span. This results from the vehicle traveling between the two clusters as shown in the analysis of the routing in § 7.2. For $120 \leq t \leq 260$, many observations and a continuous value decrease can be experienced. Here, the vehicle serves customers in the second cluster. Due to the small travel times, the value decreases minute by minute. For $t > 260$, a budget of $b = 100$ is not feasible anymore. As shown on the right side of Figure 7, $DLT(N, \sigma)$ and $DLT(N)$ show a nearly similar behavior as WLT with large intervals in the beginning and a detailed consideration in the following. $DLT(\sigma)$ shows an opposed structure. This is explained in the following.

Figure 8 shows the overall structure of $DLT(\sigma)$ after 10,000 simulation runs. The x -axis represents the point of time, the y -axis represents the time budget. As already seen in Figure 7, areas in the beginning of the time horizon are considered in detail and the entries in later areas remain unseparated. The relatively poor solution quality of $DLT(\sigma)$ compared to the other DLTs indicates that a detailed consideration in the beginning may not be beneficial compared to a detailed focus in the course of the time horizon. The single large entry in the highly separated area results from the pure exploitation of the ADP algorithm. The first observation of this entry led to a low value prohibiting further observations.

In contrast to $DLT(\sigma)$, $DLT(N)$ provides high quality solutions. The according LT-structure is shown in Figure 9. A correlation between point of time and free time budget over the time horizon can be identified. $DLT(N)$ contains three segments, approximately divided by times $t = 120$ and $t = 240$. In these points of time, the separation level indicates a low number of observations. The time budget drops significantly right afterward. This reflects a heterogeneous observation behavior over time. This behavior again results from routing. The segments are divided at times when the vehicle travels between clusters. By arriving in the new cluster, usually a large set of requests is given. The inclusions require a high amount of the free time budget leading to the drop in free time budget.

The number of entries of the different LTs after 1 million simulation runs is depicted in Table 5. For DLT, the average number of entries over parameter τ is shown. For the given instance, WLT stores the values of $|\mathcal{E}(\text{WLT})| = 59,500$ entries after 1 million simulation runs, compared to $|\mathcal{E}(\text{DLT}(N, \sigma))| = 19,700$. The number of entries is dependent on the instance's characteristics. Given a high dod and, therefore, a high initial time budget, significantly more states can be reached resulting in partitionings with many separations. Again, the number of entries for instance $\mathcal{F}_U$, $dod = 0.25$, and $\mathcal{A}_{20}$ is exceptionally small because of the high initial time budget consumption serving the ERC.

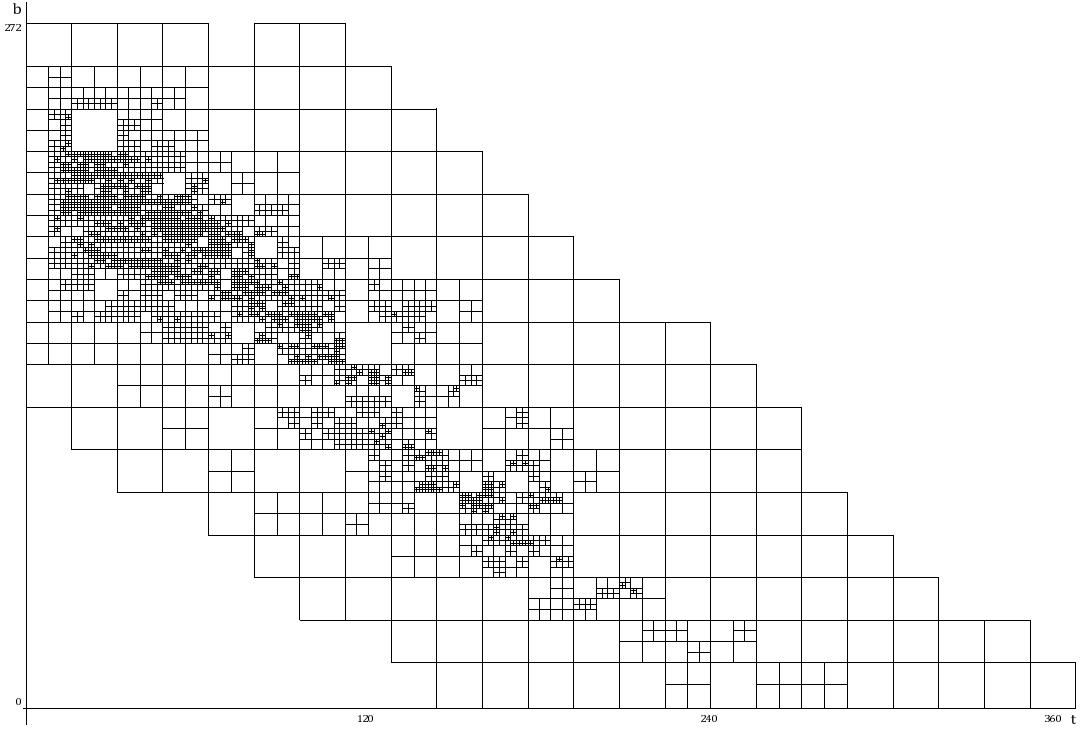


Figure 8: Structure of the $DLT(\sigma)$ with $\tau = 1.5$ for $\mathcal{F}_{2C}$, $dod = 0.75$, and $\mathcal{A}_{15}$ after 10,000 Simulation Runs

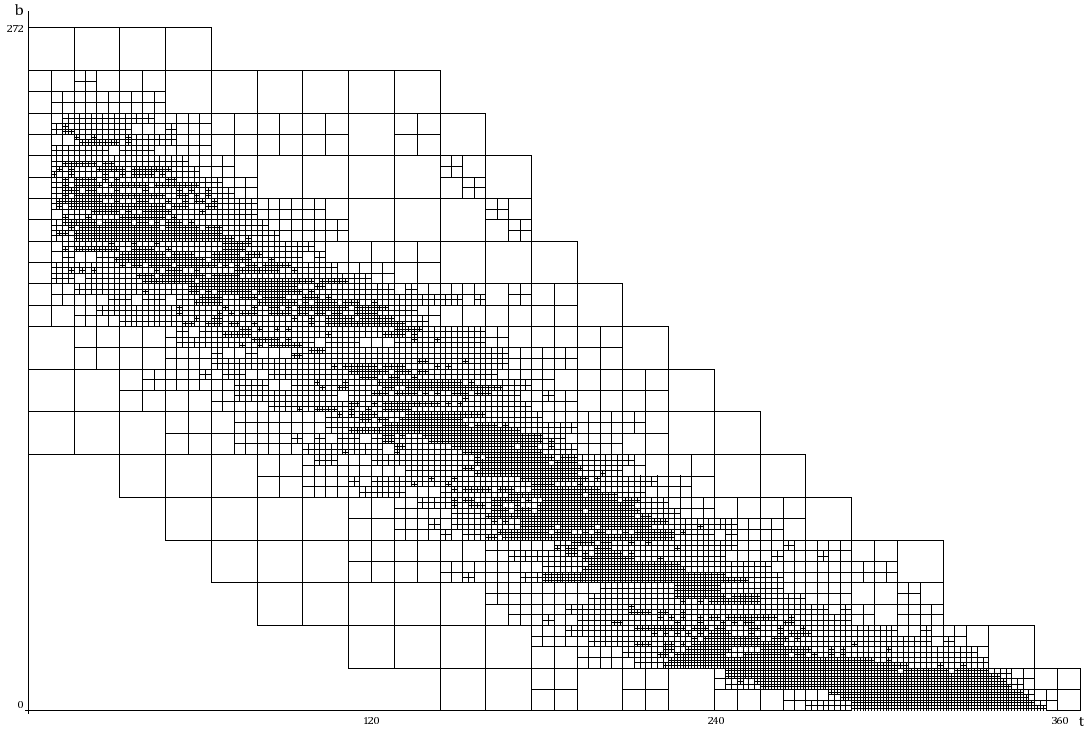
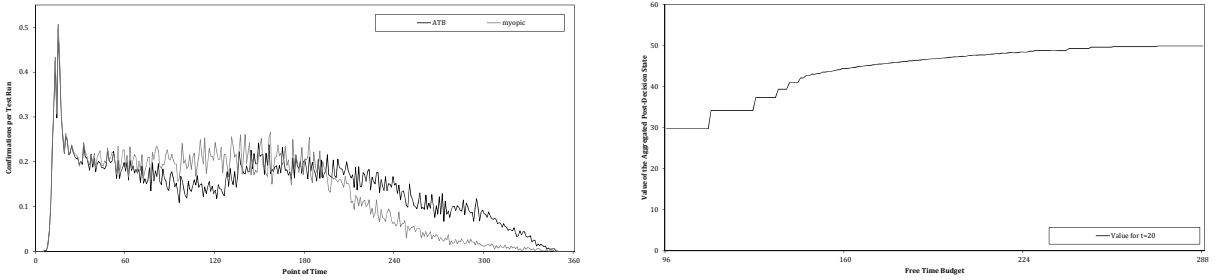


Figure 9: Structure of the $DLT(N)$ with $\tau = 1.5$ for $\mathcal{F}_{2C}$, $dod = 0.75$, and $\mathcal{A}_{20}$ after 10,000 Simulation Runs

Table 5: Number of Entries per LT and Reduction of DLT Compared to WLT

<i>dod</i>	0.25						0.50						0.75					
Service area	$\mathcal{A}_{15}$			$\mathcal{A}_{20}$			$\mathcal{A}_{15}$			$\mathcal{A}_{20}$			$\mathcal{A}_{15}$			$\mathcal{A}_{20}$		
Distribution	$\mathcal{F}_U$	$\mathcal{F}_{2C}$	$\mathcal{F}_{3C}$	$\mathcal{F}_U$	$\mathcal{F}_{2C}$	$\mathcal{F}_{3C}$	$\mathcal{F}_U$	$\mathcal{F}_{2C}$	$\mathcal{F}_{3C}$	$\mathcal{F}_U$	$\mathcal{F}_{2C}$	$\mathcal{F}_{3C}$	$\mathcal{F}_U$	$\mathcal{F}_{2C}$	$\mathcal{F}_{3C}$	$\mathcal{F}_U$	$\mathcal{F}_{2C}$	$\mathcal{F}_{3C}$
Average Number of Entries (in 1,000)																		
DLT(N, σ)	21.2	29.3	31.2	0.1	32.9	28.0	24.1	22.6	25.3	12.8	29.5	29.9	19.9	17.9	20.0	18.8	19.7	25.2
DLT(N)	15.0	22.2	23.2	1.2	25.5	20.6	18.5	22.2	23.4	7.8	25.2	24.1	19.0	19.6	20.9	13.7	19.7	22.6
DLT(σ)	21.7	23.8	25.0	8.0	23.8	25.0	20.6	20.4	22.9	17.1	23.0	26.3	19.9	17.5	18.3	21.1	19.8	23.0
WLT	47.7	60.2	64.8	20.7	60.5	54.2	59.5	57.0	61.5	38.9	64.2	64.5	62.5	52.7	58.6	59.1	59.5	65.2
Reduction to WLT (in %)																		
DLT(N, σ)	55.5	51.4	51.8	99.3	45.6	48.3	59.5	60.4	58.8	67.1	54.2	53.7	68.1	66.0	65.9	68.1	66.8	61.4
DLT(N)	68.6	63.1	64.2	94.3	57.9	61.9	69.0	61.1	61.9	79.9	60.8	62.7	69.6	62.9	64.3	76.9	66.9	65.3
DLT(σ)	54.5	60.5	61.4	61.4	60.6	53.9	65.3	64.2	62.8	56.0	64.3	59.2	68.2	66.7	68.8	64.3	66.7	64.7

Figure 10: Average Amount of Confirmations per Point of Time for ATB and *myopic*, Values for Aggregated Post-Decision State with $t = 20$

As expected, the number of entries of WLT is significantly higher than for the DLTs. The reduction of the number of entries compared to WLT is depicted in the lower part of Table 5. For the instance presented earlier, the required number of entries and the according memory consumption is reduced by 66.8%. Over all instances, this reduction reaches up to 68.1% for DLT(N, σ) and even up to 79.9% for DLT(N). Memory may nowadays be easy to access. Still, because of the exponential entry-increase by adding additional attributes to the LT, DLTs may be able to handle significantly higher-dimensional LTs compared to WLT.

7.2 Routing and Subset Selection

In the following, we analyze the decision policy achieved by ATB and the impact to subset selection and routing. We draw on the same instance presented earlier. To display the difference in confirmation behavior, the left side of Figure 10 shows the average number of confirmations per point of time for ATB and the myopic approach over 10,000 test runs. Both approaches decide nearly similar in the beginning resulting in the same number of confirmations. This can be explained by observing the values regarding free time budget in $t = 20$ as depicted on the right side of Figure 10. Between $200 \leq b \leq 260$, the values only slightly differ. This can be explained by the tradeoff between insertion times and area coverage as shown in Figure 2(a). The increase of coverage counterbalances the insertion times leading to a nearly constant value. In the following hours, the myopic heuristic confirms more customers than ATB. As expected, the number of confirmations decreases drastically in the end of the day.

Notable is the dent in confirmations for ATB in $t = 120$. This can be explained by the relatively low number of decision points, respectively customer visits, around this time. This results from the travel between the two customer clusters as assumed earlier and shown on the

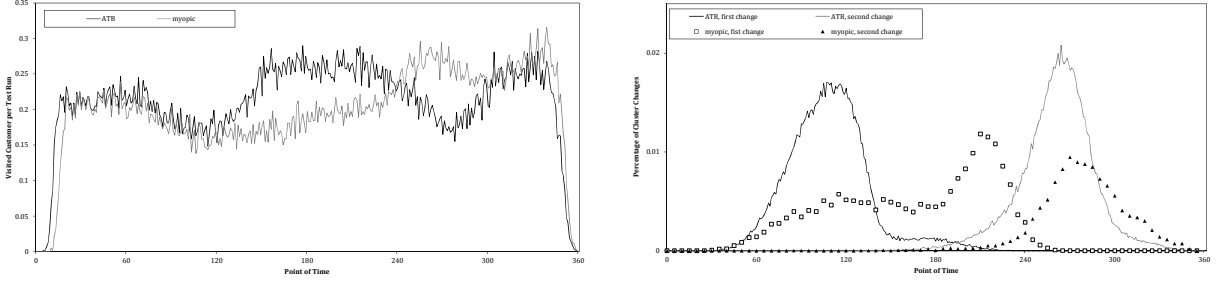


Figure 11: Average Amount of Customer Visits and Percentage of First and Second Cluster Change per Point of Time for ATB and *myopic*

left side of Figure 11. Here, the number of customer visits per point of time is displayed. For ATB, three distinguished peaks of customer visits can be discovered. This can be explained by the vehicle changing clusters. The points of time the vehicle changes clusters are depicted on the right side Figure 11. The y-axis represents the percentage of test runs, in which the vehicle left a cluster given at a certain point of time. As expected, the times of the cluster changes for the myopic heuristic vary significantly. For ATB, the first cluster change mainly occurs between $100 \leq t \leq 125$. The average travel times between the two cluster of $\mathcal{F}_{2C}$ is $d = 20$. Generally, the vehicle arrives in the second cluster before $t = 146$, as seen on the increase in visits on the left side of Figure 11. The second cluster change occurs between between the $260 \leq t \leq 270$ and is even more distinct. The anticipatory confirmation policy significantly impacts the routing decisions and determines when to change clusters. This is remarkable because no explicit spatial information is included in the ATB state evaluation.

7.3 Budgeting Time

In this section, we analyze how ATB approaches the dependencies depicted in Figure 1 and how ATB budgets the time accordingly. We compare ATB with CBH as approach with explicit subset selection and with *myopic* and AI_{WAS} . We select AI_{WAS} because the approach maximizes the coverage in the beginning. Figure 12 shows the impact of the approaches ATB, AI_{WAS} , CBH, and *myopic* on the coverage and insertion times and their decision making regarding the time budget for $\text{dod} = 0.75$ and $\mathcal{A}_{20}$. For ATB, we also show the expected number of future confirmations dependent on the free time budget for a specific point of time. The left side of Figure 12 shows the results for $\mathcal{F}_U$, the right side for $\mathcal{F}_{2C}$. Figure 12(a) depicts the development of the tour duration over time, Figure 12(b) shows the average insertion time γ_t per new request in decision point k with $t(k) = t$, calculated as depicted in Equation (20). Decision x_a confirms all requests, regardless the time limit. γ_t^a is only calculated if new requests are given in decision point k : $|\mathcal{C}_k^r| > 0$.

$$\gamma_t^a = \frac{\bar{d}(\Theta_k^{x_a}) - \bar{d}(\Theta_k)}{\mathcal{C}_k^r} \quad (20)$$

Figure 12(c) depicts the realized insertion time γ_t^c per confirmed customer and point of time as shown in Equation (21). Decision x_c is the decision selected by the regarding approach. γ_t^c is only calculated if requests are confirmed: $|\mathcal{C}_k^c| > 0$.

$$\gamma_t^c = \frac{\bar{d}(\Theta_k^{x_c}) - \bar{d}(\Theta_k)}{\mathcal{C}_k^c} \quad (21)$$

Figure 12(d) shows the development of the values regarding the free time budget for $t = 144$. We select this point of time because it lays within the arrival corridor of the vehicle in the second

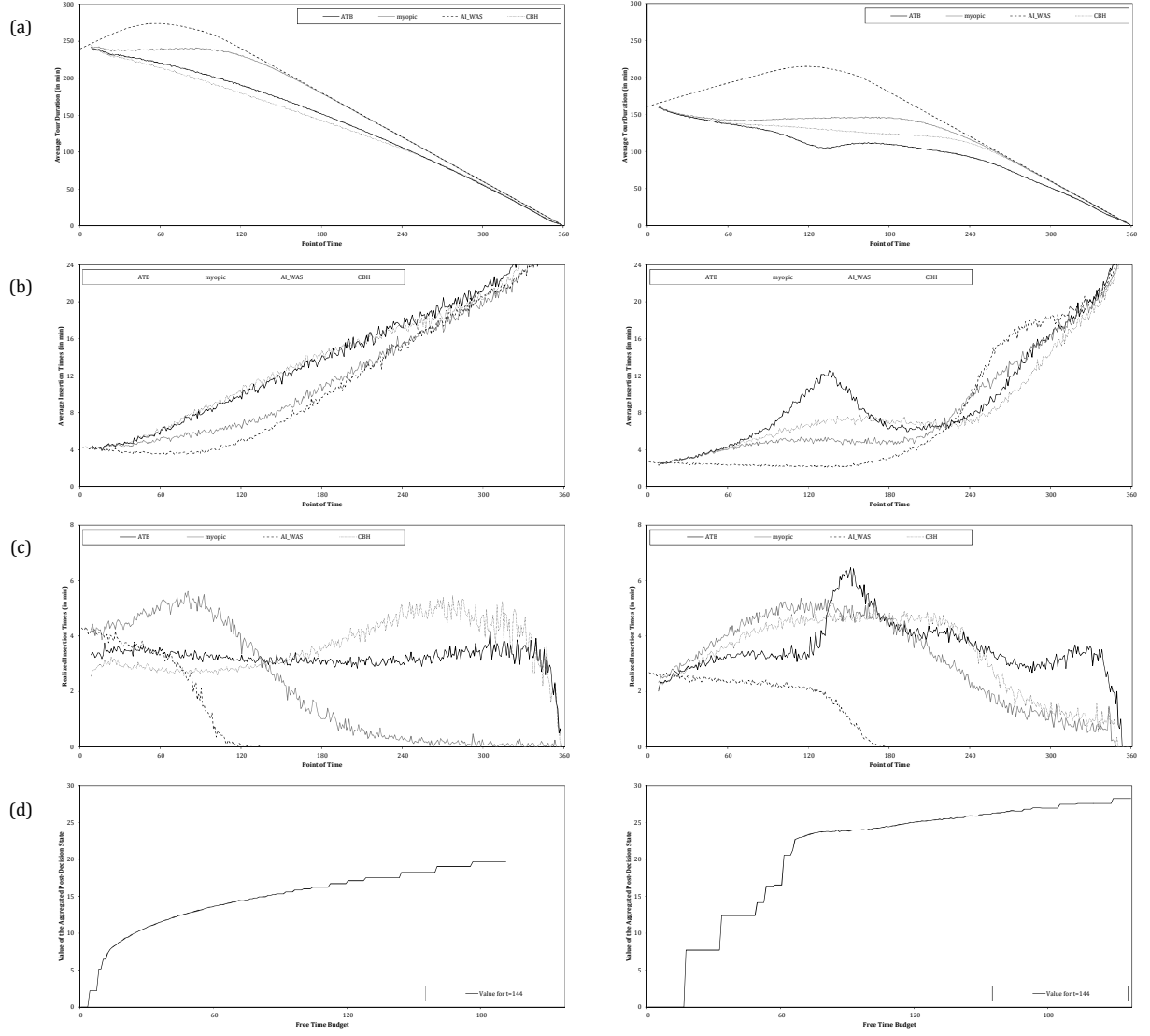


Figure 12: Tour Duration, Average and Realized Insertion Time per Customer and Time for $\mathcal{F}_U$ and $\mathcal{F}_{2C}$

cluster for $\mathcal{F}_{2C}$. For some test runs, the vehicle arrives before $t = 144$. For other test runs, the vehicle arrives after $t = 144$.

Figure 12(a) shows the average tour duration over time. As expected, approaches with explicit subset selection result in a shorter tour. AI_{WAS} and *myopic* are able to even increase area coverage over time until the free time budget is expended. Notably, the ATB tour length increases after $t = 120$ for $\mathcal{F}_{2C}$. This is the time after the first change of clusters, when many new requests are confirmed. The tour length is strongly correlated to the actual area coverage indicated by the according insertion time in Figure 12(b). AI_{WAS} and *myopic* have significantly lower average insertion times compared to ATB and CBH. The correlation of tour duration and insertion time is therefore strong. The average insertion time for $\mathcal{F}_U$ shows a continuous increase over time. For ATB and $\mathcal{F}_{2C}$, we can see a peak around $t = 135$. The insertion time for requests around this point of time is high and falls for $t > 135$. At this point of time, the vehicle served all customers in the first cluster and arrived in the second. Hence, new requests within the first cluster are highly expensive to include. The same behavior can rudimentarily be seen for CBH. Considering the realized confirmations and the regarding insertion time in Figure 12(c), we can see that for $\mathcal{F}_U$, ATB in average allows the same amount of insertion time per customer over time. AI_{WAS} and *myopic* allow a high amount in the beginning, but are not able to insert later requests because of the expended time budget. For $\mathcal{F}_{2C}$, a significant peak between $120 \leq t \leq 180$ can be seen. The highest realized insertion time is given around $t = 144$ meaning that at this point of time the vehicle usually arrived in the second cluster. Around this time, ATB identifies the requirement to spend significantly more of the free time budget to reestablish the area coverage as already depicted in the top right figure of Figure 12.

As shown in Figure 11, the vehicle arrives in the second cluster usually before $t = 146$. We now analyze how this arrival impacts the values. Figure 12(d) shows the value of a certain free time budget for $t = 144$ and $DLT(N, \sigma)$. On the x-axis, the free time budget is shown. Given $\mathcal{F}_U$, the development of the values follows the idealized example of Figure 2(d). An increase of the free time budget results in an increase of the value. Given $\mathcal{F}_{2C}$, a jump discontinuity can be seen at $b = 60$. For $b < 60$, the value drops drastically. A time budget lower than $b = 60$ reflects that the vehicle has already arrived in the second cluster. At arrival, a large subset of requests is confirmed. Hence, the post-decision state value drops. Given $b \geq 60$, the vehicle has not arrived in the cluster. A high number of confirmations can be expected and the value is significantly higher than for $b < 60$. ATB drawing on the DLT is able to identify this characteristic and, therefore, to adapt to the spatial and temporal request distribution.

8 Conclusion and Future Research

In this paper, we presented a dynamic vehicle routing problem with stochastic customer requests. The requests follow a temporal and spatial distribution and the request times and customer locations are unknown until the request occurs. We depicted that anticipation of future confirmations can be achieved by evaluating the free time budget regarding the time of day. With ATB, we developed an approach considering the area coverage and the subset selection explicitly in decision making. ATB draws on approximate dynamic programming and approximates the expected number of future confirmations regarding free time budget and point of time. For a variety of instances, ATB is able to outperform state-of-the-art benchmark heuristics.

As an offline approach, ATB allows fast decision making within the execution phase. ATB requires a lookup table to store the expected values. The LT-entries consist of intervals of point of time and free time budget. For the given problem, the required consideration of time varies during the day. The interval lengths have proven to be essential for the approximation process and the solution quality. With the dynamic lookup table, we introduced a new approach adapting

Table A1: Expected Entry Values and Deviation

Entries	p_1	p_2	p_3	p_4	$\bar{p}$
V	1.0	2.0	4.0	6.0	3.0
σ^2	0.0	1.0	3.0	4.0	4.3
ν	0.2	0.3	0.4	0.1	1.0
n_i^*	0	1,537	4,610	6,146	6,590

to the problem's and instances' specifics. A comparison to classical LT-approaches shows that the DLT is both efficient and effective providing high quality solutions and a fast approximation process.

Until now, we were not able to achieve offline approximation explicitly considering (explicit) spatial attributes in the LT. Future work may identify suitable spatial and spatial-temporal attributes to include into the LT for an explicit consideration of customers' and vehicle's locations. The work may be extended to multi-vehicle and multi-periodical settings. Rejected customers might be postponed to a following period as presented by Angelelli et al. (2009). Here, anticipatory algorithms have to consider free time budgets for several vehicles and the current and the following periods. These more complex problems require post-decision state spaces of higher dimensionality enhancing the advantages of the DLT.

Due to its generality, DLT may not only be applied to this specific set of problems but might be a valuable tool in the entire field of approximate dynamic programming. Here, the functionality of the DLT might be extended, e. g., by partitioning regarding individual parameters, by merging of entries, and by the addition and reduction of parameters depending on the approximation process.

A Appendix

In this section, we show the impact of partitioning for a simplified, single-stage example. To show the impact of the number of entries to the approximation process, we especially consider the required number of simulation runs for a sufficient approximation. We consider a partitioning in the highest level of separation resulting in four entries $\mathcal{Q} = \{p_1, \dots, p_4\}$. Additionally, we consider a partitioning $\bar{\mathcal{Q}} = \{\bar{p}\}$ containing only a single entry. In $\bar{\mathcal{Q}}$, $\bar{p}$ represents $p_1, \dots, p_4$. The value in every entry in $\mathcal{Q}$ follows a normal distribution with expected value $V(p_i)$ and standard deviation $\sigma^2(p_i)$. Additionally, let ν_i be the probability of observing p_i . The according values are shown in Table A1. The entry values behave heterogeneously, the expected values and deviations rise from p_1 to p_4 . The according values of $\bar{p}$ are a result of the partitioning.

To show the impact of the number of entries on the approximation process, we calculate the expected necessary number of observations n_i^* for every entry and the total number of simulation runs n^* for termination, i. e., for sufficient approximation in every entry. As a termination criterion, we allow a difference of the average values $\hat{V}$ to the expected values up to 0.05. Further, we calculate the number of required observations $\bar{n}$ for entry $\bar{p}$ and compare the results. For each entry, we calculate the distribution of the expected realizations average (i. e., $\alpha = \frac{1}{n}$). Then, we derive the probability $\mathbb{P}_k$ that the average lies in the allowed deviation range after k entry observations. We determine n_i^* as the minimal number of observations with probability higher than $\mathbb{P}_k > 95.0\%$. Let $\hat{V}_l(p_i)$ be the value of the l th entry realization of p_i . Then, the minimum number of observations for entry p_i can be calculated as shown in Equation (A1).

$$n_i^* = \arg \min_{k \in \mathbb{N}^+} \left\{ \mathbb{P}_k \left(\left| V(p_i) - \frac{1}{k} \sum_{l=1}^k \hat{V}_l(p_i) \right| \leq 0.05 \right) \geq 0.95 \right\} \quad (\text{A1})$$

The total number of required simulation runs n^* is the maximum number of individual observations considering the probability of observing the entry as depicted in Equation (A2).

$$n^* = \max_{i \in \{1, \dots, 4\}} \left\{ \frac{n_i^*}{\nu_i} \right\} \quad (\text{A2})$$

Rarely visited entries increase the necessary number of simulation runs of the algorithm. In the example, p_4 requires the most visits for termination with $n_4^* = 6,146$. Due to the probability $\nu_4 = 0.1$ of observing p_4 , the expected number of runs for termination of the whole process is $n^* = \frac{n_4^*}{0.1} = 61,460$. For $\mathcal{Q}$, a sufficient approximation is expected after 61,460 simulation runs.

We now show that partitioning $\bar{\mathcal{Q}}$ can reduce the number of required simulation runs significantly. The expected value of $\bar{p} \in \bar{\mathcal{Q}}$ is the weighted sum of the single expected values as depicted in Equation (A3).

$$V(\bar{p}) = \sum_{i=1}^m \nu_i V(p_i) = 3.0 \quad (\text{A3})$$

The variance $\sigma^2(\bar{p})$ can be calculated as shown in Equation (A4).

$$\sigma^2(\bar{p}) = \mathbb{E}V(\bar{p})^2 - (\mathbb{E}V(\bar{p}))^2 = \sum_{i=1}^4 \nu_i V(p_i)^2 - \left(\sum_{i=1}^4 \nu_i V(p_i) \right)^2 = 4.3 \quad (\text{A4})$$

The probability distribution of $V(\bar{p})$ is the weighted sum of the single distributions. The number of necessary observations to achieve a maximal deviation of 0.05 with a probability of at least 95% is $\bar{n} = 6,590$. The number of necessary simulation runs is reduced by 89.3% compared to $\mathcal{Q}$. For our example, partitioning $\bar{\mathcal{Q}}$ allows a significantly faster approximation. Nevertheless, the partitioning has a large impact on decision making. As we can see from Equation (A4), the variance of $V(\bar{p})$ exceeds the variance of all original entries $p_1, \dots, p_4$. Partitioning $\bar{\mathcal{Q}}$ results in a rise of the deviation of the entry value. We additionally experience a bias $|V(\bar{p}) - V(p_i)|$ up to 3.0 for all former entries. Using $\bar{p}$ results in a less accurate representation and may lead to ineffective decisions.

Evidently, partitioning $\bar{\mathcal{Q}}$ results in a suboptimal solution quality. We consider a decision point S and two possible decisions x_a, x_b . Decision x_a leads to entry p_1 , x_b to p_4 . The immediate rewards are $R(S, x_a) = 2.0$ and $R(x_b) = 1.0$. Considering the Bellman Equation, given $\mathcal{Q}$, the overall values are $R(S, x_a) + V(p_1) = 3.0$ and $R(S, x_b) + V(p_4) = 7.0$. Hence, we choose x_b and achieve an expected overall outcome of 7.0. With SLT $\bar{\mathcal{Q}}$, the two decisions result in the same entry $\bar{p}$. Hence, decision x_a is chosen with overall outcome of 3.0. Due to partitioning $\bar{\mathcal{Q}}$, we experience a loss of 4.0. In essence of the example, $\bar{\mathcal{Q}}$ allows a faster approximation, but simultaneously leads to a loss in solution quality. We experience a tradeoff between accuracy and approximation efficiency.

References

- Angelesli, Enrico, Nicola Bianchessi, Renata Mansini, Maria Grazia Speranza. 2009. Short term strategies for a dynamic multi-period routing problem. *Transportation Research Part C: Emerging Technologies* **17**(2) 106–119.
- Barto, Andrew G. 1998. *Reinforcement Learning: An Introduction*. MIT press.

- Bellman, Richard. 1956. Dynamic programming and lagrange multipliers. *Proceedings of the National Academy of Sciences of the United States of America* **42**(10) 767.
- Bellman, Richard. 1957. A markovian decision process. Tech. rep., DTIC Document.
- Bent, Russell, Pascal Van Hentenryck. 2003. Dynamic vehicle routing with stochastic requests. *IJCAI*. 1362–1363.
- Bent, Russell W., Pascal Van Hentenryck. 2004. Scenario-based planning for partially dynamic vehicle routing with stochastic customers. *Operations Research* **52**(6) 977–987.
- Bertsekas, Dimitri P., David A. Castañón. 1989. Adaptive aggregation methods for infinite horizon dynamic programming. *Automatic Control, IEEE Transactions on* **34**(6) 589–598.
- Bertsimas, Dimitris J, Garrett Van Ryzin. 1991. A stochastic and dynamic vehicle routing problem in the euclidean plane. *Operations Research* **39**(4) 601–615.
- Branke, Jürgen, Martin Middendorf, Guntram Noeth, Maged Dessouky. 2005. Waiting strategies for dynamic vehicle routing. *Transportation Science* **39**(3) 298–312.
- Campbell, Ann M. 2006. Aggregation for the probabilistic traveling salesman problem. *Computers & Operations Research* **33**(9) 2703–2724.
- Daganzo, Carlos F. 1984. Checkpoint dial-a-ride systems. *Transportation Research Part B: Methodological* **18**(4) 315–327.
- Dennis, William T. 2011. *Parcel and Small Package Delivery Industry*. Createspace.
- Ehmke, Jan F. 2012. *Integration of Information and Optimization Models for Routing in City Logistics, International Series in Operations Research & Management Science*, vol. 177. Springer.
- Ehmke, Jan F., Ann M. Campbell, Timothy L. Urban. 2015. Ensuring service levels in routing problems with time windows and stochastic travel times. *European Journal of Operational Research* **240**(2) 539–550.
- Esser, Klaus, Judith Kurte. 2015. Kep-studie 2015 - analyse des marktes in deutschland.
- Fang, Jiarui, Lei Zhao, Jan C. Fransoo, Tom Van Woensel. 2013. Sourcing strategies in supply risk management: An approximate dynamic programming approach. *Computers & Operations Research* **40**(5) 1371–1382.
- Flatberg, Truls, Geir Hasle, Oddvar Kloster, Eivind J. Nilssen, Atle Riise. 2007. Dynamic and stochastic vehicle routing in practice. *Dynamic Fleet Management*. Springer, 41–63.
- Gendreau, Michel, Francois Guertin, Jean-Yves Potvin, René Séguin. 2006. Neighborhood search heuristics for a dynamic vehicle dispatching problem with pick-ups and deliveries. *Transportation Research Part C: Emerging Technologies* **14**(3) 157–174.
- Gendreau, Michel, Francois Guertin, Jean-Yves Potvin, Eric Taillard. 1999. Parallel tabu search for real-time vehicle routing and dispatching. *Transportation Science* **33**(4) 381–390.
- George, Abraham, Warren B. Powell, Sanjeev R. Kulkarni, Sridhar Mahadevan. 2008. Value function approximation using multiple aggregation for multiattribute resource management. *Journal of Machine Learning Research* **9**(10) 2079–2111.
- Ghiani, Gianpaolo, Emanuele Manni, Antonella Quaranta, Chafi Triki. 2009. Anticipatory algorithms for same-day courier dispatching. *Transportation Research Part E: Logistics and Transportation Review* **45**(1) 96–106.
- Ghiani, Gianpaolo, Emanuele Manni, Barrett W. Thomas. 2012. A comparison of anticipatory algorithms for the dynamic and stochastic traveling salesman problem. *Transportation Science* **46**(3) 374–387.
- Goodson, Justin C., Jeffrey W. Ohlmann, Barrett W. Thomas. 2013. Rollout policies for dynamic solutions to the multivehicle routing problem with stochastic demand and duration limits. *Operations Research* **61**(1) 138–154.
- Haight, Frank A. 1967. *Handbook of the Poisson Distribution*. Wiley New York.
- Hvattum, Lars M., Arne Løkketangen, Gilbert Laporte. 2006. Solving a dynamic and stochastic vehicle routing problem with a sample scenario hedging heuristic. *Transportation Science* **40**(4) 421–438.
- Ichoua, Soumia, Michel Gendreau, Jean-Yves Potvin. 2000. Diversion issues in real-time vehicle dispatching. *Transportation Science* **34**(4) 426–438.

- Ichoua, Soumia, Michel Gendreau, Jean-Yves Potvin. 2006. Exploiting knowledge about future demands for real-time vehicle dispatching. *Transportation Science* **40**(2) 211–225.
- Kall, Peter, Stein W. Wallace. 1994. *Stochastic Programming*. John Wiley & Sons.
- Larsen, Allan, Oli B. G. Madsen, Marius M. Solomon. 2002. Partially dynamic vehicle routing-models and algorithms. *Journal of the Operational Research Society* **53**(6) 637–646.
- Lecluyse, Christophe, Tom Van Woensel, Herbert Peremans. 2009. Vehicle routing with stochastic time-dependent travel times. *4OR* **7**(4) 363–377.
- Lin, Erwin T. J., Lawrence W. Lan, Cathy S. T. Hsu. 2010. Assessing the on-road route efficiency for an air-express courier. *Journal of Advanced Transportation* **44**(4) 256–266.
- Lowe, John, Atif A. Khan, Bhakti Bhatale. 2014. Same-day delivery: Surviving and thriving in a world where instant gratification rules. Whitepaper 20, Cognizant.
- Meisel, Stephan. 2011. *Anticipatory Optimization for Dynamic Decision Making, Operations Research/Computer Science Interfaces Series*, vol. 51. Springer.
- Meisel, Stephan, Uli Suppa, Dirk C. Mattfeld. 2009. Grasp based approximate dynamic programming for dynamic routing of a vehicle. *Proceedings of VIII Metaheuristic International Conference*. Hamburg.
- Meisel, Stephan, Uli Suppa, Dirk C. Mattfeld. 2011. Serving multiple urban areas with stochastic customer requests. *Dynamics in Logistics*. Springer, 59–68.
- Mitrović-Minić, Snežana, Gilbert Laporte. 2004. Waiting strategies for the dynamic pickup and delivery problem with time windows. *Transportation Research Part B: Methodological* **38**(7) 635–655.
- Pillac, Victor, Michel Gendreau, Christelle Guéret, Andrés L. Medaglia. 2013. A review of dynamic vehicle routing problems. *European Journal of Operational Research* **225**(1) 1–11.
- Powell, Warren B. 2007. *Approximate Dynamic Programming: Solving the Curses of Dimensionality, Wiley Series in Probability and Statistics*, vol. 703. John Wiley & Sons.
- Powell, Warren B. 2009. What you should know about approximate dynamic programming. *Naval Research Logistics (NRL)* **56**(3) 239–249.
- Powell, Warren B. 2011. *Approximate Dynamic Programming: Solving the Curses of Dimensionality, Wiley Series in Probability and Statistics*, vol. 842. John Wiley & Sons.
- Powell, Warren B., Ilya O. Ryzhov. 2012. *Optimal Learning, Wiley Series in Probability and Statistics*, vol. 841. John Wiley & Sons.
- Psaraftis, Harilaos N. 1980. A dynamic programming solution to the single vehicle many-to-many immediate request dial-a-ride problem. *Transportation Science* **14**(2) 130–154.
- Rosenkrantz, Daniel J., Richard E. Stearns, Philip M. Lewis. 1974. Approximate algorithms for the traveling salesperson problem. *Switching and Automata Theory, 1974., IEEE Conference Record of 15th Annual Symposium on*. IEEE, 33–42.
- Schneeweiss, Christoph. 1999. *Hierarchies in Distributed Decision Making*. Springer Science & Business Media.
- Sungur, Ilgaz, Yingtao Ren, Fernando Ordóñez, Maged Dessouky, Hongsheng Zhong. 2010. A model and algorithm for the courier delivery problem with uncertainty. *Transportation Science* **44**(2) 193–205.
- Swihart, Michael R., Jason D. Papastavrou. 1999. A stochastic and dynamic model for the single-vehicle pick-up and delivery problem. *European Journal of Operational Research* **114**(3) 447–464.
- Tassiulas, Leandros. 1996. Adaptive routing on the plane. *Operations Research* **44**(5) 823–832.
- Thomas, Barrett W. 2007. Waiting strategies for anticipating service requests from known customer locations. *Transportation Science* **41**(3) 319–331.
- Thomas, Barrett W., Chelsea C. White III. 2004. Anticipatory route selection. *Transportation Science* **38**(4) 473–487.
- Thomas, Barrett W., Chelsea C. White III. 2007. The dynamic shortest path problem with anticipation. *European Journal of Operational Research* **176**(2) 836–854.
- Ulmer, Marlin W., Jan Brinkmann, Dirk C. Mattfeld. 2015. Anticipatory planning for courier, express and parcel services. *Logistics Management*. Springer, 313–324.
- Ulmer, Marlin W., Dirk C. Mattfeld. 2013. Modeling customers in the euclidean plane for routing applications. *Proceedings of the 14th EU/ME Workshop*. 98–103.

van Hemert, Jano I., Johannes A. La Poutré. 2004. Dynamic routing problems with fruitful regions: Models and evolutionary computation. *Parallel Problem Solving from Nature-PPSN VIII*. Springer, 692–701.